

SAPIENZA
UNIVERSITÀ DI ROMA

Fine-Tuning Large Language Models via Peer-Generated Training Data

Facoltà di Scienze Matematiche, Fisiche e Naturali
Laurea in Scienze Matematiche per l'Intelligenza Artificiale

Lorenzo Salis
Matricola 2130242

Relatore
Prof. Fabrizio Silvestri

Anno Accademico 2025/2026

Fine-Tuning Large Language Models via Peer-Generated Training Data
Sapienza Università di Roma

© 2026 Lorenzo Salis. Tutti i diritti riservati

Questa tesi è stata composta con LaTeX e la classe Sapthesis.

Email dell'autore: salis.2130242@studenti.uniroma1.it

Sommario

I Large Language Models (LLM) hanno ormai raggiunto un ruolo di primo piano nell'elaborazione del linguaggio naturale, rendendo sempre più rilevante la loro capacità di produrre risposte allineate alle aspettative degli utenti. Questo problema di *alignment* è tipicamente affrontato tramite Reinforcement Learning from Human Feedback (RLHF), il cui costo e la cui scarsa scalabilità motivano la ricerca di alternative automatiche alla raccolta di annotazioni umane.

Il presente lavoro valuta la fattibilità del fine-tuning tramite Direct Preference Optimization (DPO) a partire da dataset di preferenze generati automaticamente dal framework di valutazione reciproca AutoBench, senza alcuna annotazione umana. A partire dai punteggi collettivi prodotti da AutoBench sono stati costruiti dataset di preferenze secondo due strategie di estrazione (Best-vs-Worst e Pairwise), utilizzati per il fine-tuning di due modelli open-source di dimensioni contenute, Gemma 2 2B Instruct e Qwen 2.5 0.5B Instruct, in un ambiente computazionale limitato.

I modelli fine-tunati sono stati valutati su un test set interno derivato da AutoBench e su benchmark pubblici standardizzati. I risultati mostrano che l'approccio è tecnicamente realizzabile e preserva le capacità generali dei modelli, e che la qualità delle coppie di preferenza incide più della loro quantità sulle prestazioni finali. Per Qwen 2.5 0.5B si osserva inoltre un bias di lunghezza nelle log-probabilità utilizzate per il calcolo dell'accuracy di preferenza, che ne penalizza sistematicamente la valutazione anche dopo normalizzazione; il fenomeno residuo viene discusso alla luce di un'ipotesi di *typicality bias*.

Indice

1 Introduzione **1**

2 Background teorico **3**
2.1 Large Language Models . . . 3
2.1.1 L'architettura Transformer . . . 3
2.1.2 Pre-training . . . 4
2.1.3 Instruction tuning . . . 5
2.1.4 Limiti dei modelli pre-addestrati . . . 5
2.2 Fine-tuning dei Large Language Models . . . 5
2.2.1 Full fine-tuning . . . 6
2.2.2 Parameter-Efficient Fine-Tuning . . . 6
2.2.3 Low-Rank Adaptation (LoRA) . . . 6
2.2.4 QLoRA . . . 7
2.3 Alignment mediante preferenze . . . 8
2.3.1 Reinforcement Learning from Human Feedback . . . 8
2.3.2 Il modello di ricompensa (Reward Model) . . . 9
2.3.3 Proximal Policy Optimization (PPO) . . . 10
2.3.4 Limiti di RLHF . . . 10
2.4 Direct Preference Optimization . . . 11
2.4.1 Idea alla base di DPO . . . 11
2.4.2 Struttura del dataset: prompt, chosen e rejected . . . 12
2.4.3 Vantaggi rispetto a RLHF . . . 12
2.4.4 Perché DPO è adatto al presente lavoro . . . 13
2.5 Peer-generated training data . . . 13
2.5.1 LLM-as-a-Judge . . . 13
2.5.2 Peer assessment . . . 14
2.5.3 Dataset di preferenze automatici . . . 14
2.5.4 Vantaggi e limiti . . . 14
2.5.5 Introduzione ad AutoBench . . . 15

3 AutoBench e costruzione dei dataset di preferenze **16**
3.1 AutoBench come sorgente di dati peer-generated . . . 16
3.2 Preparazione e pulizia dei dati . . . 17
3.3 Estrazione delle coppie di preferenza . . . 19
3.3.1 Strategia Best-vs-Worst . . . 20
3.3.2 Strategia Pairwise . . . 21

3.4 Filtri di qualità 21
3.4.1 Filtro sul divario di preferenza 21
3.4.2 Filtro di similarità testuale 22
3.4.3 Limitazione del numero di coppie per prompt 22
3.5 Costruzione dei dataset finali 23
3.5.1 Dataset Best-vs-Worst 23
3.5.2 Dataset Pairwise 23
3.5.3 Confronto tra i dataset 24
3.6 Suddivisione dei dati 25

4 Configurazione sperimentale **27**
4.1 Obiettivi sperimentali 27
4.2 Ambiente sperimentale 27
4.3 Procedura di fine-tuning 28
4.3.1 Quantizzazione e QLoRA 29
4.3.2 Configurazione dell'addestramento 29
4.3.3 Modelli considerati 30
4.4 Configurazione degli esperimenti 30
4.5 Metodologia di valutazione 32
4.5.1 Valutazione sul test set AutoBench 32
4.5.2 Accuracy grezza e accuracy normalizzata 32
4.5.3 Benchmark esterni 33
4.5.4 Framework di valutazione 33

5 Risultati sperimentali **35**
5.1 Introduzione ai risultati 35
5.2 Gemma 2 2B + Dataset Best-Worst 35
5.2.1 Risultati sul test set AutoBench 36
5.2.2 Risultati sui benchmark esterni 36
5.3 Qwen 2.5 0.5B + Dataset Best-Worst 37
5.3.1 Risultati sul test set AutoBench 37
5.3.2 Risultati sui benchmark esterni 38
5.4 Qwen 2.5 0.5B + Dataset Pairwise ($\Delta = 0.3$) 38
5.4.1 Risultati sul test set AutoBench 38
5.4.2 Risultati sui benchmark esterni 39
5.5 Qwen 2.5 0.5B + Dataset Pairwise ($\Delta = 0.5$) 39
5.5.1 Risultati sul test set AutoBench 40
5.5.2 Risultati sui benchmark esterni 40
5.6 Sintesi dei risultati 41

6 Discussione dei risultati **42**
6.1 Valutazione dell'efficacia dei peer-generated training data 42
6.2 Influenza della qualità del dataset di preferenze 43
6.3 Interpretazione critica dei risultati sperimentali 44
6.4 Bias osservati durante la valutazione 45
6.4.1 Bias di lunghezza 45
6.4.2 Un'ipotesi aggiuntiva: bias di tipicità 46

6.5 Limiti del presente lavoro 47
6.6 Prospettive future 49

7 Conclusioni **51**

Capitolo 1

Introduzione

Negli ultimi anni i *Large Language Models* (LLM) hanno assunto un ruolo centrale nell'elaborazione automatica del linguaggio naturale, dimostrando capacità notevoli in un'ampia varietà di compiti linguistici. Accanto ai progressi nelle architetture e nelle procedure di addestramento, una sfida fondamentale riguarda l'*alignment*: la capacità di orientare il comportamento di questi modelli verso risposte coerenti con le preferenze e le aspettative degli utenti. Il paradigma più consolidato per affrontare questo problema, il Reinforcement Learning from Human Feedback (RLHF), si basa sulla raccolta di giudizi espressi da annotatori umani, un processo che garantisce qualità elevata ma risulta costoso, lento e difficilmente scalabile.

Questa limitazione ha motivato un crescente interesse verso metodi automatici per la costruzione di dati di preferenza, nei quali il giudizio umano viene sostituito da quello di altri modelli linguistici. Il presente lavoro si inserisce in questo filone di ricerca, esplorando la fattibilità di un approccio interamente basato su *peer-generated training data*: dati di preferenza derivati dalla valutazione reciproca tra modelli linguistici, senza il ricorso ad annotazioni umane.

A tale scopo viene utilizzato AutoBench, un framework di valutazione automatica nel quale i modelli partecipanti assumono simultaneamente il ruolo di generatori e valutatori, producendo punteggi che riflettono un giudizio collettivo sulla qualità delle risposte. I dati così ottenuti vengono trasformati in dataset di preferenze nel formato richiesto da Direct Preference Optimization (DPO), un algoritmo di allineamento che consente di aggiornare direttamente i parametri di un modello linguistico a partire da confronti tra risposte, evitando la necessità di addestrare separatamente un modello di ricompensa. Gli esperimenti sono condotti su due modelli open-source di dimensioni contenute, Gemma 2 2B Instruct e Qwen 2.5 0.5B Instruct, in un ambiente computazionale volutamente limitato, al fine di verificare la praticabilità dell'approccio anche al di fuori di contesti con risorse elevate.

I contributi principali del presente lavoro sono i seguenti:

- la progettazione e l'implementazione di una pipeline completa per la trasformazione dei dati grezzi di AutoBench in dataset di preferenze compatibili con l'addestramento tramite DPO, comprensiva di procedure di filtraggio e deduplicazione;

- la costruzione e il confronto di diverse strategie di estrazione delle preferenze (Best-vs-Worst e Pairwise con soglie differenti), con analisi dell'impatto della qualità del segnale di preferenza sulle prestazioni finali;
- una valutazione sperimentale sistematica dei modelli fine-tunati sia su un test set interno derivato da AutoBench sia su benchmark pubblici standardizzati;
- l'osservazione e l'analisi dell'effetto del bias di lunghezza delle log-probabilità non normalizzate sull'accuracy di preferenza osservata per Qwen 2.5 0.5B, evidenziando alcune implicazioni metodologiche per la valutazione automatica mediante dataset di preferenze.

Struttura della tesi

Il Capitolo 2 introduce i concetti teorici necessari alla comprensione del lavoro: l'architettura Transformer, le tecniche di fine-tuning efficiente (LoRA e QLoRA), il paradigma RLHF e le sue limitazioni, l'algoritmo DPO e i principali approcci alla costruzione automatica di dati di preferenza. Il Capitolo 3 descrive AutoBench come sorgente di dati *peer-generated* e illustra la pipeline di costruzione dei dataset di preferenze, dalle operazioni di pulizia e normalizzazione fino alle strategie di estrazione e ai filtri di qualità adottati. Il Capitolo 4 presenta la configurazione sperimentale: modelli utilizzati, ambiente computazionale, iperparametri di addestramento e metodologia di valutazione. Il Capitolo 5 presenta i risultati ottenuti nei diversi esperimenti, analizzando le prestazioni dei modelli sui differenti dataset di preferenze. Il Capitolo 6 discute i risultati nel contesto più ampio della letteratura, ne evidenzia i limiti e propone possibili direzioni di ricerca futura. Il Capitolo 7 conclude il lavoro con una sintesi dei contributi e delle osservazioni principali.

Capitolo 2

Background teorico

2.1 Large Language Models

L'elaborazione automatica del linguaggio naturale (*Natural Language Processing*, NLP) è stata profondamente trasformata dall'affermarsi dei Large Language Models (LLM), che hanno raggiunto prestazioni senza precedenti in un'ampia varietà di compiti linguistici. Modelli come GPT, Llama, Gemma e Qwen sono oggi in grado di comprendere e generare testo naturale, rispondere a domande, riassumere documenti, tradurre tra lingue differenti e assistere gli utenti in numerose attività complesse.

Questa versatilità deriva dalla combinazione di tre fattori: architetture neurali di grandi dimensioni, enormi quantità di dati di addestramento e risorse computazionali su scala crescente. A differenza dei precedenti sistemi di NLP, progettati per svolgere un singolo compito specifico, gli LLM apprendono una rappresentazione generale del linguaggio che può essere successivamente adattata a molteplici applicazioni, spesso senza la necessità di un addestramento dedicato per ciascuna di esse.

2.1.1 L'architettura Transformer

La quasi totalità dei moderni Large Language Models si basa sull'architettura *Transformer*, introdotta da Vaswani et al. [1]. Tale architettura ha segnato un punto di svolta rispetto alle precedenti reti neurali ricorrenti, grazie all'introduzione del meccanismo di *self-attention*, che consente al modello di analizzare simultaneamente tutte le posizioni di una sequenza, superando i vincoli di elaborazione sequenziale tipici dei modelli ricorrenti.

Concettualmente, per ogni token in input vengono calcolate tre rappresentazioni distinte — *query* (Q), *key* (K) e *value* (V) — e l'output dell'attenzione è ottenuto come combinazione pesata dei value, dove i pesi sono determinati dalla similarità tra query e key:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \tag{2.1}$$

dove d_k è la dimensione dei vettori key, utilizzata come fattore di normalizzazione. Questo meccanismo permette al modello di assegnare un peso differente ai diversi token del contesto durante la generazione di una nuova parola, catturando efficacemente relazioni sia locali sia a lunga distanza all'interno del testo.

Un ulteriore vantaggio del Transformer rispetto alle architetture ricorrenti riguarda l'addestramento: poiché l'attenzione opera simultaneamente su tutte le posizioni della sequenza, le operazioni di training risultano altamente parallelizzabili, rendendo possibile la costruzione di modelli con miliardi di parametri su hardware moderno. Per queste ragioni il Transformer costituisce oggi l'architettura di riferimento per la maggior parte dei modelli linguistici di grandi dimensioni.

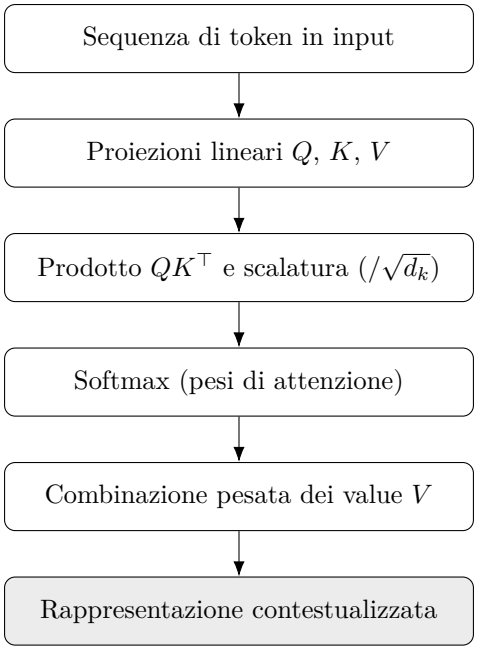


Figura 2.1. Rappresentazione schematica del meccanismo di self-attention: dalla sequenza in input vengono ricavate le proiezioni Q, K e V; il prodotto $QK^\top$, opportunamente riscalato e normalizzato tramite softmax, determina i pesi con cui i value V vengono combinati per produrre l'output contestualizzato.

2.1.2 Pre-training

La fase iniziale dell'addestramento di un Large Language Model è nota come *pre-training*. In questa fase il modello viene addestrato su enormi collezioni di testi provenienti da libri, pagine web, articoli scientifici e altre sorgenti testuali, con l'obiettivo di apprendere le regolarità statistiche del linguaggio naturale.

L'obiettivo di addestramento più comune è la predizione del token successivo all'interno di una sequenza, formalizzabile come la massimizzazione della log-verosimiglianza dei dati di addestramento:

$$\mathcal{L}_{\text{pre-training}}(\theta) = \sum_{i=1}^{N} \log P_\theta(t_i \mid t_1, \ldots, t_{i-1}) \tag{2.2}$$

dove $t_1, \ldots, t_N$ è la sequenza di token e θ i parametri del modello. Ripetendo questo processo su miliardi di esempi, il modello acquisisce progressivamente conoscenze sintattiche, semantiche e, in parte, fattuali, sviluppando una rappresentazione interna

del linguaggio sufficientemente generale da poter essere riutilizzata in numerosi contesti differenti.

Al termine del pre-training il modello possiede quindi ampie capacità linguistiche, ma non è ancora specificamente ottimizzato per seguire istruzioni o interagire efficacemente con un utente: la sua naturale propensione è infatti la continuazione plausibile di un testo, non la risposta a una richiesta.

2.1.3 Instruction tuning

Per rendere i modelli adatti alle applicazioni conversazionali viene generalmente eseguita una seconda fase di addestramento, nota come *instruction tuning*. In questa fase il modello viene ulteriormente addestrato su dataset costituiti da coppie istruzione-risposta, con l'obiettivo di apprendere a seguire correttamente le richieste formulate dagli utenti piuttosto che limitarsi a continuare il testo in input.

Rispetto al pre-training, l'instruction tuning impiega dataset di dimensioni significativamente inferiori, ma caratterizzati da dati accuratamente selezionati o annotati. Questa procedura trasforma un modello linguistico generico in un modello *instruction-following*, capace di produrre risposte più pertinenti, coerenti e utili durante l'interazione.

I modelli impiegati nel presente lavoro, Gemma 2 2B Instruct e Qwen 2.5 0.5B Instruct, appartengono a questa categoria e costituiscono il punto di partenza degli esperimenti di fine-tuning descritti nei capitoli successivi.

2.1.4 Limiti dei modelli pre-addestrati

Pre-training e instruction tuning, per quanto efficaci, non garantiscono che le risposte generate siano sempre corrette, affidabili o coerenti tra loro. Un modello può produrre informazioni inesatte (*hallucinations*), contraddirsi all'interno della stessa conversazione, oppure privilegiare formulazioni fluenti ma poco accurate dal punto di vista fattuale.

Queste limitazioni sono in parte intrinseche agli obiettivi di addestramento descritti in precedenza: né la predizione del token successivo né l'instruction tuning supervisionato forniscono al modello un segnale esplicito su *quanto* una risposta sia utile, sicura o preferibile rispetto a un'alternativa — distinguono cioè una continuazione plausibile da una buona risposta, ma non quantificano la qualità relativa tra più risposte corrette. Colmare questo gap è l'obiettivo delle tecniche di *alignment* basate sulle preferenze, discusse nella Sezione 2.3.

2.2 Fine-tuning dei Large Language Models

Come discusso nella Sezione 2.1, il pre-training consente ai Large Language Models di acquisire una conoscenza generale del linguaggio naturale, mentre l'instruction tuning ne adatta il comportamento all'esecuzione di istruzioni. I modelli risultanti da queste due fasi non sono tuttavia necessariamente ottimizzati per compiti specifici o per particolari domini applicativi: per questo motivo si rende spesso necessaria un'ulteriore fase di adattamento, nota come *fine-tuning*.

Il fine-tuning consiste nell'aggiornamento dei parametri di un modello pre-addestrato utilizzando un dataset specifico per il compito di interesse, evitando così di ripetere il costoso processo di pre-training. Nel contesto dei Large Language Models, il fine-tuning viene impiegato per numerose finalità, tra cui l'adattamento a domini specialistici, il miglioramento della capacità di seguire istruzioni e, come vedremo nella Sezione 2.3, l'allineamento del comportamento del modello alle preferenze degli utenti. Le tecniche presentate in questa sezione costituiscono gli strumenti metodologici impiegati anche in quel contesto: ciò che cambia, nei capitoli successivi, è la natura del segnale di addestramento — non più una singola risposta corretta, ma un confronto tra risposte alternative.

2.2.1 Full fine-tuning

L'approccio più diretto al fine-tuning consiste nell'aggiornare tutti i parametri del modello durante l'addestramento. Questa strategia, comunemente denominata *full fine-tuning*, permette al modello di adattarsi completamente al nuovo compito, ma presenta un costo computazionale molto elevato.

I moderni Large Language Models possono contenere da centinaia di milioni a centinaia di miliardi di parametri: l'aggiornamento completo di tali modelli richiede GPU ad alte prestazioni, grandi quantità di memoria video e tempi di addestramento considerevoli, rendendo questa soluzione spesso impraticabile in contesti di ricerca con risorse limitate. Inoltre, per ogni nuovo compito è necessario memorizzare una copia completa dei pesi del modello, con un conseguente aumento dello spazio di archiviazione richiesto.

2.2.2 Parameter-Efficient Fine-Tuning

Per superare queste limitazioni sono state sviluppate diverse tecniche di *Parameter-Efficient Fine-Tuning* (PEFT), il cui obiettivo è ridurre il numero di parametri aggiornati durante l'addestramento mantenendo prestazioni comparabili al full fine-tuning.

L'idea alla base di questi approcci consiste nel mantenere congelati i pesi del modello pre-addestrato e aggiornare esclusivamente un numero ridotto di parametri aggiuntivi. In questo modo diminuiscono sia il consumo di memoria sia il tempo necessario per il training, rendendo possibile il fine-tuning anche su hardware relativamente contenuto. Tra le numerose famiglie di tecniche PEFT proposte in letteratura — che comprendono, ad esempio, approcci basati su *prompt tuning* o su moduli aggiuntivi (*adapter layers*) — quella attualmente più diffusa per i Large Language Models è *Low-Rank Adaptation* (LoRA), descritta nella sezione seguente.

2.2.3 Low-Rank Adaptation (LoRA)

LoRA, introdotta da Hu et al. [2], si basa sull'osservazione che l'aggiornamento dei pesi necessario per adattare un modello a un nuovo compito possiede tipicamente un rango intrinseco molto basso, anche quando le matrici dei pesi originali hanno dimensioni elevate.

Sulla base di questa osservazione, invece di aggiornare direttamente una matrice dei pesi $W_0 \in \mathbb{R}^{d \times k}$, LoRA la mantiene congelata e introduce un aggiornamento

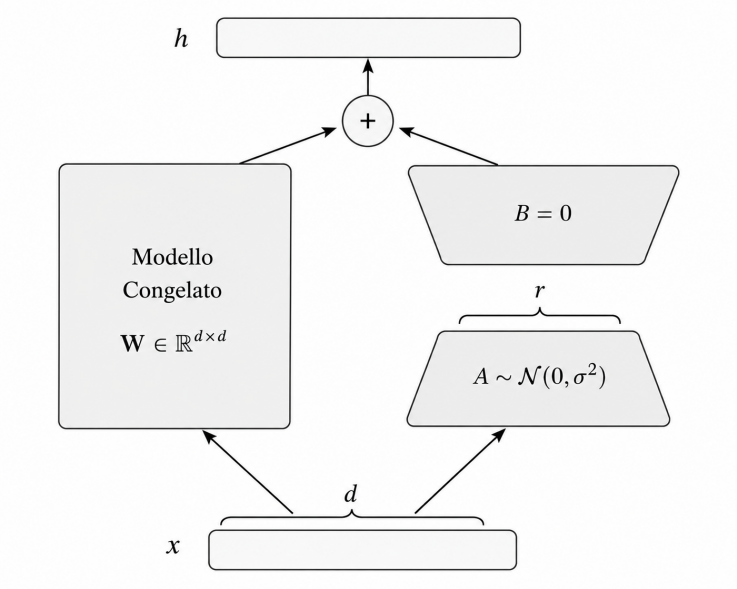


Figura 2.2. Schema del funzionamento di LoRA. I pesi pre-addestrati $\mathbf{W}$ rimangono congelati, mentre l'aggiornamento $\Delta W = BA$ viene appreso mediante due matrici di rango ridotto r. La matrice A viene inizializzata casualmente, mentre B viene inizializzata a zero. L'output finale h è ottenuto sommando il contributo del modello pre-addestrato e dell'adattamento LoRA.

ΔW espresso come prodotto di due matrici di rango ridotto r, detti *adapter*:

$$W = W_0 + \Delta W = W_0 + BA, \qquad B \in \mathbb{R}^{d \times r}, \;\; A \in \mathbb{R}^{r \times k}, \;\; r \ll \min(d, k) \tag{2.3}$$

Durante il training solo i parametri di A e B vengono aggiornati, mentre W_0 rimane congelato. Poiché il rango r è generalmente molto inferiore alla dimensione delle matrici originali, il numero di parametri allenabili si riduce drasticamente: nella pratica è spesso sufficiente aggiornare meno dell'1% dei parametri complessivi del modello per ottenere risultati competitivi rispetto al full fine-tuning.

Un ulteriore vantaggio di questo approccio risiede nella possibilità di distribuire separatamente gli adapter A e B, di dimensioni ridotte, evitando di dover memorizzare o condividere una copia completa del modello per ciascun compito appreso.

2.2.4 QLoRA

QLoRA (Quantized Low-Rank Adaptation), proposta da Dettmers et al. [3], estende l'approccio di LoRA combinandolo con la quantizzazione del modello base, riducendo ulteriormente i requisiti di memoria necessari per il fine-tuning.

Nello specifico, QLoRA mantiene i pesi del modello pre-addestrato in formato quantizzato a 4 bit (tipicamente nel formato *NormalFloat4*, NF4), mentre gli adapter LoRA vengono mantenuti e aggiornati in precisione più elevata. Durante il *forward pass* i pesi quantizzati vengono temporaneamente dequantizzati per il calcolo, ma rimangono memorizzati in formato compresso, riducendo significativamente il consu-

mo di memoria video rispetto a un caricamento del modello in precisione piena o mista.

Questa strategia rende possibile il fine-tuning di modelli contenenti miliardi di parametri anche su GPU con memoria limitata, come le GPU T4 utilizzate per gli esperimenti descritti nei capitoli successivi. Nel presente lavoro tutti gli esperimenti di fine-tuning sono stati effettuati mediante QLoRA, utilizzando la libreria Unsloth [4] per il caricamento dei modelli quantizzati e la gestione degli adapter LoRA; i dettagli implementativi e gli iperparametri utilizzati saranno descritti nel Capitolo 4.

2.3 Alignment mediante preferenze

Come discusso nella Sezione 2.1.4, pre-training e instruction tuning non forniscono al modello un segnale esplicito sulla qualità relativa di risposte alternative. L'alignment è il processo mediante il quale il comportamento di un modello viene modificato affinché le risposte generate risultino maggiormente coerenti con le preferenze umane, agendo così proprio sulla dimensione lasciata scoperta dalle fasi di addestramento precedenti.

L'obiettivo dell'alignment non consiste quindi nell'aumentare la quantità di conoscenza appresa dal modello, bensì nel guidarne il comportamento verso risposte considerate più corrette, utili e sicure. A differenza del tradizionale apprendimento supervisionato, nel quale per ogni input è disponibile una singola risposta corretta, i problemi di allineamento vengono generalmente affrontati attraverso *dataset di preferenze*. In questo tipo di dataset, per uno stesso prompt sono presenti due o più risposte confrontate tra loro; un annotatore, umano oppure automatico, indica quale risposta sia preferibile secondo determinati criteri di qualità.

Le preferenze così raccolte costituiscono un segnale di apprendimento più ricco rispetto al semplice testo di riferimento, poiché consentono al modello di apprendere non soltanto *cosa* rispondere, ma anche *quale* risposta sia da considerarsi migliore tra diverse alternative. Il paradigma che ha maggiormente contribuito a diffondere questo approccio è il *Reinforcement Learning from Human Feedback* (RLHF), descritto nella sezione seguente.

2.3.1 Reinforcement Learning from Human Feedback

L'idea alla base di RLHF consiste nell'utilizzare il giudizio umano come segnale di apprendimento. Dopo le fasi di pre-training e instruction tuning descritte nella Sezione 2.1, il modello viene impiegato per generare più risposte a partire dallo stesso prompt. Tali risposte vengono successivamente confrontate da annotatori umani, ai quali viene richiesto di indicare quale risposta risulti preferibile secondo criteri quali correttezza, completezza, chiarezza e sicurezza.

Le preferenze raccolte vengono quindi utilizzate per costruire una pipeline di ottimizzazione articolata in due fasi successive: l'addestramento di un modello di ricompensa (*reward model*), descritto nella Sezione 2.3.2, e una successiva fase di reinforcement learning basata sull'algoritmo Proximal Policy Optimization (PPO), finalizzata all'aggiornamento del modello linguistico e descritta nella Sezione 2.3.3.

La Figura 2.3 riassume schematicamente il flusso operativo del paradigma RLHF.

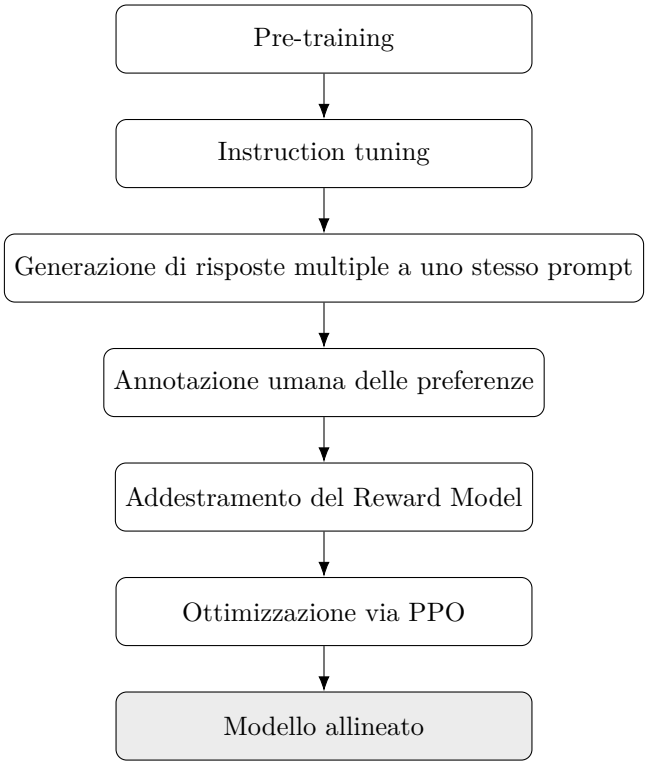


Figura 2.3. Flusso operativo del paradigma Reinforcement Learning from Human Feedback (RLHF): a partire dal modello pre-addestrato e sottoposto a instruction tuning, vengono generate risposte multiple successivamente annotate da valutatori umani; le preferenze raccolte vengono usate per addestrare un reward model, il cui segnale guida infine l'ottimizzazione del modello linguistico tramite PPO.

2.3.2 Il modello di ricompensa (Reward Model)

Per poter guidare l'aggiornamento del modello linguistico in funzione delle preferenze raccolte, RLHF prevede l'addestramento di un modello ausiliario, denominato *reward model*, il cui compito è assegnare un punteggio scalare a ciascuna risposta generata, rendendo così esplicito e quantificabile il giudizio di qualità espresso dagli annotatori.

Il reward model viene tipicamente ottenuto a partire dal modello linguistico stesso, sostituendo l'ultima testa di generazione del vocabolario con una testa di regressione che produce un singolo valore scalare. Il suo addestramento si basa sui confronti a coppie raccolti durante l'annotazione: data una coppia di risposte (y_w, y_l) generate a partire dallo stesso prompt x, dove y_w indica la risposta preferita (*winner*) e y_l quella scartata (*loser*), il modello viene addestrato affinché y_w riceva un punteggio più alto di y_l.

Tale obiettivo viene formalizzato attraverso il modello di Bradley-Terry [5], secondo cui la probabilità che y_w sia preferita a y_l è funzione della differenza tra i rispettivi punteggi:

$$P(y_w \succ y_l \mid x) = \sigma\big(r_\phi(x, y_w) - r_\phi(x, y_l)\big) \tag{2.4}$$

dove r_ϕ indica il reward model parametrizzato da ϕ e σ la funzione sigmoide. I

parametri ϕ vengono ottimizzati minimizzando la *negative log-likelihood* di tale probabilità sull'intero dataset di preferenze.

Una volta addestrato, il reward model viene utilizzato come funzione di ricompensa nella successiva fase di reinforcement learning, fornendo al modello linguistico un segnale di apprendimento che approssima il giudizio umano senza richiedere l'intervento diretto di annotatori per ogni nuova risposta generata.

2.3.3 Proximal Policy Optimization (PPO)

Una volta disponibile il reward model, la fase finale di RLHF consiste nell'aggiornare i parametri del modello linguistico — in questo contesto denominato *policy* — in modo da massimizzare la ricompensa attesa secondo il reward model addestrato. Questo passaggio viene generalmente affrontato mediante algoritmi di reinforcement learning, tra i quali il più diffuso è *Proximal Policy Optimization* (PPO), proposto da Schulman et al. [6].

PPO è un algoritmo on-policy che aggiorna la policy in maniera incrementale, limitando l'ampiezza degli aggiornamenti ad ogni iterazione mediante un obiettivo surrogato vincolato (*clipped surrogate objective*). Tale meccanismo previene variazioni eccessive della policy tra un aggiornamento e il successivo, contribuendo a stabilizzare un processo di addestramento tipicamente instabile quando applicato a spazi di azione di grandi dimensioni come quelli dei Large Language Models.

Nel contesto di RLHF, l'obiettivo ottimizzato da PPO non si limita alla massimizzazione della ricompensa fornita dal reward model, ma include generalmente un termine di penalizzazione basato sulla divergenza di Kullback-Leibler (KL) tra la policy aggiornata e una policy di riferimento, solitamente coincidente con il modello ottenuto al termine dell'instruction tuning:

$$\max_{\pi_\theta} \; \mathbb{E}_{x,\, y \sim \pi_\theta}\big[r_\phi(x, y)\big] - \beta \, \mathrm{KL}\big(\pi_\theta(y \mid x) \,\|\, \pi_{\mathrm{ref}}(y \mid x)\big) \tag{2.5}$$

dove π_θ è la policy in addestramento, π_{ref} la policy di riferimento e β un coefficiente che regola l'intensità della penalizzazione. Questo termine impedisce che il modello si discosti eccessivamente dal comportamento linguistico appreso nelle fasi precedenti pur di massimizzare la ricompensa — fenomeno noto come *reward hacking*.

L'addestramento mediante PPO richiede quindi il mantenimento simultaneo in memoria di più modelli: la policy in addestramento, la policy di riferimento, il reward model e, spesso, un ulteriore modello di valore (*value model*) utilizzato per stimare la funzione di vantaggio. Tale requisito comporta un notevole incremento della complessità implementativa e del costo computazionale complessivo della pipeline.

2.3.4 Limiti di RLHF

Nonostante l'efficacia dimostrata in numerosi sistemi conversazionali, RLHF presenta diverse limitazioni che ne complicano l'adozione, specialmente in contesti con risorse computazionali limitate.

In primo luogo, la pipeline risulta computazionalmente onerosa: come discusso nella Sezione 2.3.3, la fase di reinforcement learning richiede il mantenimento simultaneo di più modelli di dimensioni paragonabili, con un conseguente aumento significativo del consumo di memoria rispetto a un singolo modello.

In secondo luogo, il processo richiede una generazione *online* di risposte da parte della policy durante l'addestramento, le quali devono essere successivamente valutate dal reward model. Tale necessità introduce ulteriore overhead computazionale e rende l'intera procedura più lenta rispetto ai tradizionali approcci di fine-tuning supervisionato.

Infine, l'addestramento tramite reinforcement learning è noto per la sua instabilità: piccole variazioni negli iperparametri possono condurre a risultati molto differenti, e il processo è generalmente più sensibile a problemi quali il reward hacking o il collasso della policy rispetto a tecniche di ottimizzazione più dirette.

Tali limitazioni hanno motivato la ricerca di approcci alternativi in grado di ottenere risultati di alignment comparabili a RLHF, riducendo al contempo la complessità implementativa e i requisiti computazionali. Tra questi, una delle proposte più rilevanti è la *Direct Preference Optimization* (DPO), descritta nella sezione successiva.

2.4 Direct Preference Optimization

Come discusso nella Sezione 2.3.4, le limitazioni di RLHF — costo computazionale, necessità di generazione online e instabilità dell'addestramento tramite reinforcement learning — hanno motivato la ricerca di approcci alternativi capaci di ottenere risultati di alignment comparabili, riducendo al contempo la complessità implementativa. La *Direct Preference Optimization* (DPO), proposta da Rafailov et al. [7], rappresenta una delle soluzioni più rilevanti in questa direzione.

2.4.1 Idea alla base di DPO

L'osservazione centrale alla base di DPO riguarda l'obiettivo di ottimizzazione vincolato impiegato in PPO (Sezione 2.3.3):

$$\max_{\pi_\theta} \; \mathbb{E}_{x,\, y\sim\pi_\theta}\big[r_\phi(x,y)\big] - \beta\, \mathrm{KL}\big(\pi_\theta(y \mid x) \,\|\, \pi_{\mathrm{ref}}(y \mid x)\big)$$

Tale obiettivo, per una funzione di ricompensa r fissata, ammette una soluzione in forma chiusa. La policy ottimale π^* che lo massimizza è infatti esprimibile come:

$$\pi^*(y \mid x) = \frac{1}{Z(x)}\, \pi_{\mathrm{ref}}(y \mid x)\, \exp\!\left(\frac{1}{\beta} r(x,y)\right) \tag{2.6}$$

dove $Z(x) = \sum_y \pi_{\mathrm{ref}}(y \mid x) \exp\!\big(\frac{1}{\beta} r(x,y)\big)$ è una costante di normalizzazione che dipende esclusivamente dal prompt x, e non dalla risposta y.

Invertendo questa relazione è possibile esprimere la ricompensa in funzione della policy ottimale:

$$r(x,y) = \beta \log \frac{\pi^*(y \mid x)}{\pi_{\mathrm{ref}}(y \mid x)} + \beta \log Z(x) \tag{2.7}$$

Questo passaggio è cruciale: la ricompensa può essere riscritta interamente in termini della policy stessa, senza la necessità di un modello di ricompensa separato. Sostituendo questa espressione nel modello di Bradley-Terry introdotto

nella Sezione 2.3.2, il termine $Z(x)$ si elide nella differenza tra le ricompense di y_w e y_l (poiché dipende solo da x, identico per entrambe le risposte), ottenendo:

$$P(y_w \succ y_l \mid x) = \sigma\left(\beta \log \frac{\pi^*(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi^*(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right) \tag{2.8}$$

Questa espressione lega direttamente la probabilità di preferenza alla policy, senza fare più alcun riferimento esplicito a una funzione di ricompensa. DPO sfrutta esattamente questa proprietà: invece di addestrare un reward model e successivamente ottimizzare la policy tramite reinforcement learning, è possibile addestrare la policy π_θ direttamente sui dati di preferenza, minimizzando la negative log-likelihood della relazione precedente:

$$\mathcal{L}_{\text{DPO}}(\theta) = -\,\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}\left[\log \sigma\left(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right)\right] \tag{2.9}$$

L'addestramento si riduce così a un problema di classificazione binaria su coppie di risposte, risolvibile con un singolo passaggio di ottimizzazione supervisionata, analogamente a quanto avviene nel fine-tuning standard.

2.4.2 Struttura del dataset: prompt, chosen e rejected

Coerentemente con la formulazione precedente, un dataset di preferenze per DPO è costituito da terne (x, y_w, y_l), dove x è il prompt, y_w la risposta preferita e y_l quella scartata. Nella letteratura e nelle implementazioni più diffuse — incluse le librerie utilizzate nel presente lavoro — questi elementi vengono comunemente indicati come *prompt*, *chosen* e *rejected*, terminologia che verrà adottata nei capitoli successivi.

È importante osservare che questa struttura coincide esattamente con quella richiesta per l'addestramento del reward model in RLHF (Sezione 2.3.2): in entrambi i casi si parte da confronti a coppie tra risposte, annotati da un giudice umano o automatico. La differenza tra i due approcci non risiede quindi nella natura dei dati richiesti, bensì nel modo in cui tali dati vengono utilizzati: mentre RLHF li impiega per addestrare un modello di ricompensa intermedio, DPO li utilizza per aggiornare direttamente i parametri della policy.

2.4.3 Vantaggi rispetto a RLHF

La riformulazione proposta da DPO comporta diversi vantaggi pratici rispetto alla pipeline RLHF descritta nella Sezione 2.3.

In primo luogo, DPO elimina la necessità di addestrare un reward model separato: la policy π_θ viene aggiornata direttamente sui dati di preferenza, riducendo il numero di modelli da mantenere in memoria rispetto all'insieme descritto nella Sezione 2.3.3. DPO necessita infatti soltanto della policy in addestramento e della policy di riferimento — quest'ultima peraltro utilizzabile in modalità di sola inferenza, senza calcolo del gradiente.

In secondo luogo, DPO non richiede alcuna generazione *online* di risposte durante l'addestramento: l'intera procedura opera in modalità offline su un dataset di preferenze fissato a priori, in modo analogo a un addestramento supervisionato

standard. Questo elimina l'overhead associato al campionamento iterativo dalla policy, caratteristico di PPO.

Infine, poiché l'obiettivo di DPO si riduce a una funzione di perdita differenziabile in forma chiusa, l'addestramento risulta più stabile e semplice da implementare rispetto a un ciclo di reinforcement learning, evitando problematiche quali il reward hacking o il collasso della policy discusse nella Sezione 2.3.4. Rafailov et al. [7] riportano inoltre prestazioni comparabili o superiori a RLHF su diversi compiti di allineamento, a fronte di una complessità implementativa nettamente inferiore.

2.4.4 Perché DPO è adatto al presente lavoro

Le caratteristiche di DPO discusse in questa sezione lo rendono particolarmente adatto al contesto sperimentale del presente lavoro, caratterizzato da risorse computazionali limitate e dall'utilizzo di modelli di dimensioni contenute.

In primo luogo, l'assenza di un reward model separato e di un ciclo di generazione online riduce sensibilmente i requisiti di memoria video, un vincolo particolarmente stringente nell'ambiente Kaggle con GPU T4 impiegato per gli esperimenti descritti nel Capitolo 4. In secondo luogo, DPO si combina naturalmente con le tecniche di fine-tuning efficiente descritte nella Sezione 2.2: poiché la policy in addestramento è l'unico modello che richiede l'aggiornamento dei parametri, è possibile applicare QLoRA direttamente al suo addestramento, mantenendo invariati i vantaggi in termini di memoria discussi in precedenza.

Infine, e in modo più diretto, DPO richiede come unico input un dataset di preferenze nella forma (prompt, chosen, rejected). I giudizi relativi prodotti da AutoBench attraverso la valutazione reciproca tra modelli, pur non essendo nativamente strutturati in questa forma, si prestano naturalmente a essere trasformati in triple di preferenza tramite la pipeline di estrazione descritta nella Sezione 2.5, senza richiedere l'addestramento di un modello di ricompensa intermedio o altre elaborazioni aggiuntive. Questa compatibilità tra la natura dei dati disponibili e i requisiti dell'algoritmo di allineamento ha motivato la scelta di DPO come metodo di fine-tuning per gli esperimenti descritti nei Capitoli 4 e 5.

2.5 Peer-generated training data

Come discusso nella Sezione 2.4, l'addestramento tramite DPO richiede un dataset di preferenze nella forma (prompt, chosen, rejected). Nella formulazione originale di RLHF, queste annotazioni vengono raccolte da valutatori umani: un processo che garantisce un giudizio di alta qualità, ma che risulta costoso, lento e difficilmente scalabile a dataset di grandi dimensioni. Per questo motivo, negli ultimi anni si è affermato un crescente interesse verso metodi automatici per la costruzione di dataset di preferenze, in cui il giudizio umano viene parzialmente o totalmente sostituito da quello di altri modelli linguistici.

2.5.1 LLM-as-a-Judge

Il paradigma *LLM-as-a-Judge* consiste nell'utilizzare un Large Language Model per valutare le risposte generate da altri modelli, in sostituzione del giudizio umano.

Zheng et al. [8] hanno mostrato come modelli di grandi dimensioni, opportunamente istruiti tramite prompt, siano in grado di valutare la qualità di risposte testuali con un livello di accordo con i giudizi umani comparabile a quello osservabile tra annotatori umani diversi.

Questo approccio si basa tipicamente sull'impiego di un singolo modello "giudice", solitamente di dimensioni e capacità superiori rispetto ai modelli generatori, al quale viene richiesto di confrontare due o più risposte secondo criteri espliciti (correttezza, completezza, utilità) e di indicare quale risulti preferibile. Il vantaggio principale di questo approccio risiede nella sua scalabilità: una volta definito il prompt di valutazione, è possibile annotare automaticamente un numero arbitrariamente grande di confronti, senza i costi e i tempi associati all'annotazione umana.

2.5.2 Peer assessment

Una variante di questo paradigma, denominata *peer assessment* o *peer review*, prevede che la valutazione delle risposte non sia affidata a un singolo modello giudice di capacità superiore, ma a uno o più modelli di scala comparabile a quelli che hanno generato le risposte stesse — talvolta gli stessi modelli generatori, chiamati a valutare reciprocamente le proprie produzioni. Li et al. [9] hanno proposto un framework di questo tipo, mostrando come la combinazione dei giudizi di più modelli "alla pari" (*peer rank*), eventualmente integrata da una fase di discussione tra i modelli stessi, possa mitigare alcuni bias tipici della valutazione affidata a un singolo giudice.

Questo approccio si distingue da LLM-as-a-Judge in senso stretto per l'assenza di un modello giudice qualitativamente superiore ai generatori: la valutazione emerge invece dal confronto reciproco tra modelli di competenza paragonabile, in analogia con i processi di revisione tra pari (*peer review*) impiegati in ambito accademico.

2.5.3 Dataset di preferenze automatici

L'impiego di giudici automatici, secondo uno dei due paradigmi descritti, consente di costruire dataset di preferenze su larga scala senza il ricorso ad annotatori umani. Un esempio rilevante è UltraFeedback [10], un dataset di preferenze costruito raccogliendo risposte generate da numerosi modelli a un ampio insieme di prompt e valutandole automaticamente tramite un modello giudice, secondo criteri multipli quali utilità, onestà e innocuità.

Questi dataset condividono una struttura comune: un insieme di prompt, un insieme di modelli generatori che producono risposte candidate, e una procedura — automatica o semi-automatica — che confronta tali risposte e ne deriva coppie etichettate (chosen, rejected), direttamente utilizzabili per l'addestramento tramite DPO.

2.5.4 Vantaggi e limiti

L'impiego di giudici automatici per la costruzione di dataset di preferenze presenta vantaggi significativi in termini di scalabilità e costo. È infatti possibile generare un numero arbitrariamente grande di confronti senza i tempi e i costi associati al reclutamento e alla formazione di annotatori umani, rendendo praticabile la

costruzione di dataset di preferenze anche in contesti di ricerca con risorse limitate, come quello del presente lavoro.

Questo approccio presenta tuttavia alcune limitazioni. In primo luogo, i giudizi automatici sono soggetti a bias documentati in letteratura, quali la tendenza a preferire risposte più lunghe indipendentemente dalla loro qualità effettiva (*verbosity bias*), una sensibilità all'ordine in cui le risposte vengono presentate (*position bias*), oppure, quando il modello giudice appartiene alla stessa famiglia di uno dei generatori, una tendenza a preferire risposte stilisticamente simili alle proprie (*self-preference bias*). In secondo luogo, la qualità complessiva del dataset risultante dipende interamente dalla qualità del giudice impiegato: eventuali errori o bias sistematici nella valutazione si propagano direttamente nelle preferenze annotate, e quindi nel comportamento appreso dal modello durante il fine-tuning. Infine, nel caso specifico della *peer assessment*, l'assenza di un giudice qualitativamente superiore ai generatori può comportare il rischio che eventuali limiti condivisi tra i modelli coinvolti (ad esempio bias stilistici comuni o lacune di conoscenza) non vengano rilevati, in quanto non riconosciuti come tali da nessuno dei valutatori.

2.5.5 Introduzione ad AutoBench

Il presente lavoro si inserisce in questo filone di ricerca, adottando il paradigma della *peer assessment* per la costruzione del dataset di preferenze impiegato negli esperimenti di fine-tuning. A tale scopo è stato utilizzato *AutoBench* [11], un framework di valutazione automatica nel quale un insieme di modelli linguistici genera, valuta e confronta reciprocamente le rispettive risposte, producendo una misura aggregata della loro qualità.

Oltre a fornire una classifica dei modelli partecipanti, AutoBench rende disponibili le valutazioni assegnate alle singole risposte. Tali informazioni costituiscono la base per la costruzione automatica di dataset di preferenze, trasformando i punteggi ottenuti durante il processo di *peer assessment* in coppie `(chosen, rejected)` compatibili con algoritmi di allineamento basati su preferenze, come Direct Preference Optimization.

Nel presente lavoro i dati prodotti da AutoBench sono stati sottoposti a una fase di preprocessing finalizzata alla costruzione dei dataset di addestramento utilizzati negli esperimenti. I dettagli relativi al framework e all'intera pipeline di preparazione dei dati sono descritti nel Capitolo 3.

Capitolo 3

AutoBench e costruzione dei dataset di preferenze

Come discusso nel Capitolo 2, il presente lavoro utilizza AutoBench [11] come sorgente dei dati di preferenza impiegati per il fine-tuning dei modelli linguistici. Sebbene il framework sia stato originariamente progettato come benchmark di valutazione automatica basato sul paradigma della *peer assessment*, le informazioni prodotte durante il processo di valutazione possono essere riutilizzate per costruire dataset di preferenze compatibili con algoritmi di allineamento quali Direct Preference Optimization (DPO).

Il presente capitolo descrive la struttura dei dati forniti da AutoBench e la pipeline sviluppata per trasformare tali dati in dataset di addestramento. Vengono inoltre illustrate le procedure di pulizia, filtraggio, deduplicazione ed estrazione delle coppie di preferenza adottate nel presente lavoro.

3.1 AutoBench come sorgente di dati peer-generated

L'addestramento di modelli linguistici mediante tecniche di allineamento basate sulle preferenze, come Direct Preference Optimization (DPO), richiede la disponibilità di dataset contenenti coppie di risposte ordinate secondo una relazione di preferenza. Tradizionalmente tali dataset vengono costruiti attraverso processi di annotazione umana, nei quali uno o più valutatori selezionano, per ciascun prompt, la risposta ritenuta migliore tra diverse alternative. Sebbene questo approccio garantisca elevata qualità, esso risulta costoso in termini di tempo e risorse economiche, limitando la possibilità di generare dataset di grandi dimensioni.

Nel presente lavoro si è scelto di esplorare un approccio alternativo basato sull'utilizzo di dati di preferenza generati automaticamente da altri modelli linguistici. In particolare, è stato utilizzato AutoBench, un framework di valutazione automatica nel quale gli LLM partecipanti assumono simultaneamente il ruolo di generatori di task, concorrenti e valutatori. Attraverso un processo di valutazione reciproca, ciascun modello assegna un punteggio alle risposte prodotte dagli altri partecipanti, contribuendo alla costruzione di una classifica collettiva delle prestazioni.

L'aspetto di maggiore interesse per questa tesi non risiede nel ranking finale prodotto da AutoBench, bensì nei dati generati durante il processo di valutazione.

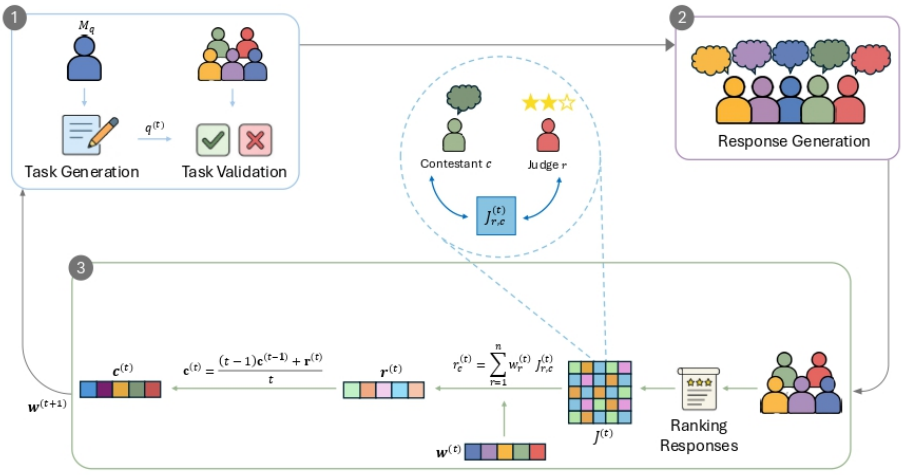


Figura 3.1. Schema del processo di valutazione adottato da AutoBench: generazione e validazione del task, generazione delle risposte da parte dei modelli contestant e valutazione reciproca da parte dei modelli giudice, con aggiornamento iterativo dei pesi di giudizio. Figura riprodotta da [11].

Per ogni prompt sono infatti disponibili le risposte fornite dai diversi modelli, insieme ai punteggi ottenuti attraverso il meccanismo di *peer assessment*. Tali informazioni costituiscono una sorgente naturale di dati di preferenza, poiché consentono di identificare risposte giudicate migliori o peggiori rispetto ad altre senza ricorrere ad annotazioni umane aggiuntive.

A partire da queste osservazioni, il dataset originale di AutoBench è stato reinterpretato come una raccolta di preferenze implicite tra risposte generate da modelli differenti. L'obiettivo del processo descritto in questo capitolo consiste quindi nella trasformazione dei dati grezzi prodotti da AutoBench in dataset compatibili con l'addestramento mediante Direct Preference Optimization, mantenendo il più possibile il segnale di preferenza derivante dalla valutazione collettiva dei modelli partecipanti.

3.2 Preparazione e pulizia dei dati

Il dataset ottenuto da AutoBench contiene, per ciascun prompt valutato, le risposte generate dai modelli partecipanti insieme ai relativi punteggi derivanti dal processo di *peer assessment*. Prima di procedere all'estrazione delle coppie di preferenza, è stato necessario effettuare una fase preliminare di preparazione e pulizia dei dati, finalizzata a garantire la qualità e la consistenza delle informazioni utilizzate per il fine-tuning.

In primo luogo, sono stati mantenuti esclusivamente i modelli *open-source* presenti nel benchmark. Questa scelta è stata motivata dalla volontà di costruire un dataset interamente derivato da modelli liberamente accessibili e riproducibili, evitando l'influenza di sistemi proprietari non disponibili alla comunità di ricerca. A seguito della rimozione delle risposte generate dai modelli *closed-source*, sono stati eliminati dai dizionari dei giudizi anche i relativi punteggi espressi durante la fase di valutazione.

Tabella 3.1. Modelli open-source selezionati dalla run AutoBench utilizzata in questa tesi.

Famiglia	Modello
DeepSeek	DeepSeek-R1-0528
DeepSeek	DeepSeek-v3.2
DeepSeek	DeepSeek-v3.2-speciale
GLM	GLM-4.5-Air
GLM	GLM-4.6
Kimi	Kimi-k2-0905
Kimi	Kimi-k2-thinking
Nemotron	Llama-3.3-Nemotron-Super-49B-v1.5
MiniMax	MiniMax-M2
Mistral	Mistral-Large-2512
Mistral	Mistral-Medium-3.1
Mistral	Ministral-8B-2512
Nemotron	Nemotron-3-Nano-30B-A3B
Nemotron	Nemotron-Nano-9B-v2
Olmo	Olmo-3.1-32B-Think
Qwen	Qwen3-235B-A22B-Thinking-2507
Qwen	Qwen3-235B-A22B-2507
Qwen	Qwen3-Next-80B-A3B-Thinking
GPT-OSS	GPT-OSS-20B
GPT-OSS	GPT-OSS-120B

Non essendo disponibile l'algoritmo originale di aggregazione utilizzato da AutoBench, il punteggio finale di ciascuna risposta è stato ricalcolato come media aritmetica dei soli giudizi provenienti dai modelli *open-source* rimanenti.

Nel framework AutoBench, ciascun modello valutatore assegna alle risposte concorrenti un punteggio su una scala discreta da 1 a 5, dove i valori più alti indicano una qualità percepita superiore. Il punteggio medio così ottenuto, indicato nel dataset dalla colonna `average_rank_os`, costituisce la base per le successive fasi di estrazione delle coppie di preferenza descritte nella Sezione 3.3.

La Tabella 3.1 riporta l'elenco completo dei modelli mantenuti nella fase di costruzione dei dataset.

Successivamente è stata effettuata una fase di normalizzazione testuale mediante la libreria `ftfy` (*Fixes Text For You*), utilizzata per correggere automaticamente eventuali problemi di codifica dei caratteri presenti nel dataset. In particolare, sono stati individuati e corretti casi di *mojibake*, ovvero sequenze di caratteri corrotte generate da errori di conversione tra differenti codifiche testuali. Tale operazione ha permesso di uniformare il contenuto delle colonne contenenti prompt e risposte, riducendo il rischio di introdurre rumore durante le successive fasi di elaborazione. La Figura 3.2 mostra alcuni esempi rappresentativi delle correzioni effettuate.

Sono state inoltre rimosse tutte le istanze caratterizzate da informazioni mancanti nelle colonne essenziali (`question`, `answer` e `average_rank_os`). Per limitare ulteriormente la presenza di esempi anomali, è stato applicato un filtro sulla lunghezza delle risposte. In particolare, sono state mantenute esclusivamente le risposte

Esempi di correzione del *mojibake* tramite `ftfy`

Prima
```
- Drive allâ€reduce *P99* latency down...
```
Dopo
```
- Drive all-reduce *P99* latency down...
```

Prima
```
...Metaâ€s 10k-GPU...
```
Dopo
```
...Meta's 10k-GPU...
```

Prima
```
...medium-sized â€œelephantâ€ gradient flows...
```
Dopo
```
...medium-sized "elephant" gradient flows...
```

Figura 3.2. Esempi di correzione automatica di errori di codifica presenti nel dataset.

contenenti un numero di parole compreso tra 3 e 2000. Tale scelta ha consentito di escludere sia risposte vuote o estremamente brevi, sia generazioni eccezionalmente lunghe potenzialmente riconducibili a errori o comportamenti anomali dei modelli.

Al termine di questa fase è stato ottenuto un insieme di dati pulito e coerente, utilizzato come base per la successiva estrazione delle coppie di preferenza necessarie all'addestramento mediante Direct Preference Optimization.

Inoltre, l'obiettivo del lavoro consiste nel valutare la possibilità di generare dati di addestramento a partire esclusivamente da risorse aperte, rendendo l'intera pipeline replicabile senza dipendere da modelli proprietari.

La Figura 3.3 riassume schematicamente le fasi di preparazione e pulizia dei dati appena descritte, dal dataset grezzo prodotto da AutoBench fino all'insieme di dati pulito utilizzato come base per l'estrazione delle coppie di preferenza.

3.3 Estrazione delle coppie di preferenza

L'obiettivo finale del processo di preparazione dei dati consiste nella costruzione di dataset compatibili con l'addestramento mediante *Direct Preference Optimization* (DPO). Tale metodologia richiede, per ogni esempio di addestramento, una tripla composta da un prompt, una risposta preferita (*chosen*) e una risposta non preferita (*rejected*). Poiché il dataset originario di AutoBench contiene molteplici risposte associate allo stesso prompt e i relativi punteggi derivanti dal processo di valutazione tra pari, è stato necessario definire una procedura per convertire tali informazioni in coppie di preferenza esplicite.

Nel presente lavoro sono state esplorate due differenti strategie di estrazione delle preferenze. Entrambe si basano sull'ipotesi che il punteggio medio assegnato da AutoBench rappresenti una stima affidabile della qualità relativa delle risposte

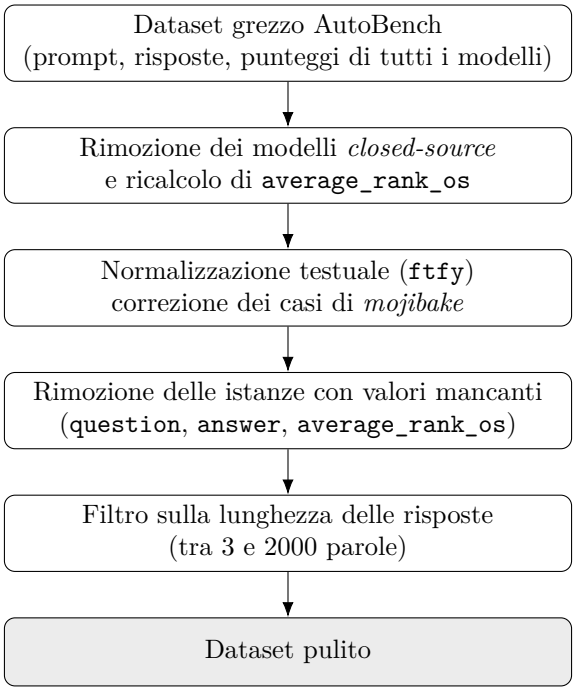


Figura 3.3. Pipeline di preparazione e pulizia dei dati: a partire dal dataset grezzo prodotto da AutoBench vengono rimossi i modelli *closed-source*, corretti gli errori di codifica testuale, eliminate le istanze con valori mancanti nelle colonne essenziali e filtrate le risposte in base alla lunghezza, ottenendo il dataset pulito utilizzato nelle fasi successive.

e possa quindi essere utilizzato per stabilire una relazione di preferenza tra due generazioni differenti.

3.3.1 Strategia Best-vs-Worst

La prima strategia, denominata "Best-vs-Worst", prevede la selezione di una sola coppia di preferenza per ciascun prompt. Per ogni insieme di risposte associate allo stesso quesito viene individuata la risposta caratterizzata dal punteggio medio più elevato e quella caratterizzata dal punteggio medio più basso. La prima viene assegnata al campo *chosen*, mentre la seconda viene assegnata al campo *rejected*.

Formalmente, dato un insieme di risposte $R = \{r_1, r_2, ..., r_n\}$ associate a un prompt q, con relativi punteggi s_i, vengono selezionate:

$$r_{chosen} = \arg\max_{r_i \in R} s_i$$

$$r_{rejected} = \arg\min_{r_i \in R} s_i$$

Questa strategia produce un insieme di dati relativamente compatto, ma caratterizzato da un segnale di preferenza particolarmente forte, poiché massimizza la distanza tra la risposta considerata migliore e quella considerata peggiore per ciascun prompt.

3.3.2 Strategia Pairwise

La seconda strategia, denominata "Pairwise", mira invece a sfruttare una porzione più ampia delle informazioni contenute nel ranking prodotto da AutoBench. Per ogni prompt vengono considerate tutte le possibili coppie di risposte generate dai modelli partecipanti.

Data una coppia di risposte (r_i, r_j), viene confrontato il rispettivo punteggio medio. La risposta con punteggio maggiore viene assegnata al campo *chosen*, mentre quella con punteggio minore viene assegnata al campo *rejected*.

La differenza tra i punteggi delle due risposte viene definita come:

$$\Delta_{ij} = |s_i - s_j|$$

Le coppie ottenute vengono successivamente sottoposte ai filtri di qualità descritti nella Sezione 3.4.

L'approccio Pairwise consente di ottenere un numero significativamente maggiore di esempi rispetto alla strategia Best-vs-Worst, aumentando la quantità di dati disponibili per il fine-tuning. Tuttavia, esso richiede ulteriori meccanismi di controllo qualità per evitare la presenza di coppie ridondanti o scarsamente informative, descritti nella sezione successiva.

3.4 Filtri di qualità

L'applicazione diretta delle strategie descritte nella sezione precedente produce un elevato numero di coppie di preferenza, non tutte ugualmente informative ai fini dell'addestramento. In particolare, alcune coppie possono essere caratterizzate da differenze di qualità trascurabili, altre possono contenere risposte quasi identiche tra loro, mentre altre ancora possono risultare sovrarappresentate a causa dell'elevato numero di combinazioni generate a partire da uno stesso prompt.

Per mitigare tali problematiche sono stati introdotti diversi meccanismi di filtraggio, con l'obiettivo di incrementare la qualità complessiva del dataset e preservare un segnale di preferenza il più possibile significativo.

3.4.1 Filtro sul divario di preferenza

Per ogni coppia candidata viene calcolata la differenza assoluta tra i punteggi medi assegnati da AutoBench alle due risposte:

$$\Delta = |s_{chosen} - s_{rejected}|$$

Le coppie caratterizzate da un valore di Δ inferiore a una soglia prefissata vengono scartate.

Poiché i punteggi sono espressi su una scala da 1 a 5, un valore di $\Delta = 0.3$ rappresenta una differenza relativamente contenuta, mentre $\Delta = 0.5$ corrisponde a un divario più marcato tra le valutazioni medie ottenute dalle due risposte.

L'introduzione di questo filtro è motivata dall'osservazione che differenze molto ridotte tra i punteggi possono riflettere situazioni nelle quali le due risposte risultano sostanzialmente equivalenti secondo il giudizio collettivo espresso dai modelli valutatori.

La soglia Δ costituisce pertanto un parametro di controllo della qualità del dataset: valori più elevati privilegiano coppie caratterizzate da preferenze più nette, mentre valori inferiori consentono di aumentare il numero di esempi disponibili a discapito della forza del segnale di apprendimento.

3.4.2 Filtro di similarità testuale

Anche in presenza di differenze significative nei punteggi, due risposte possono risultare estremamente simili dal punto di vista del contenuto. In tali casi la coppia fornisce un'informazione limitata al modello, poiché la distinzione tra risposta preferita e risposta non preferita risulta poco evidente.

Per identificare queste situazioni è stata utilizzata la similarità di Jaccard calcolata sull'insieme delle parole contenute nelle due risposte:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

dove A e B rappresentano gli insiemi delle parole presenti nelle due generazioni confrontate.

Le coppie caratterizzate da una similarità superiore a una soglia prefissata sono state eliminate dal dataset. Tale procedura consente di ridurre la presenza di esempi ridondanti e di favorire l'apprendimento a partire da confronti realmente distintivi.

Esempio illustrativo di coppia esclusa per elevata similarità testuale

Prompt: Spiega la differenza principale tra una `list` e una `tuple` in Python.

Risposta A (chosen): In Python, una list è una struttura dati mutabile, ovvero i suoi elementi possono essere modificati, aggiunti o rimossi dopo la creazione. Una tuple, invece, è immutabile: una volta creata, il suo contenuto non può essere alterato.

Risposta B (rejected): In Python, una tuple è una struttura dati immutabile, ovvero il suo contenuto non può essere alterato dopo la creazione. Una list, invece, è mutabile: i suoi elementi possono essere modificati, aggiunti o rimossi.

Similarità di Jaccard calcolata: $J \approx 0.91$ *(soglia di esclusione: 0.85)*

Figura 3.4. Esempio illustrativo di una coppia di risposte semanticamente equivalenti, che differiscono solo nell'ordine di presentazione dei concetti. Nonostante un possibile divario nei punteggi assegnati da AutoBench, l'elevata similarità lessicale porta all'esclusione della coppia dal dataset finale.

3.4.3 Limitazione del numero di coppie per prompt

Nel caso della strategia Pairwise, il numero di coppie generate a partire da un singolo prompt cresce rapidamente all'aumentare del numero di risposte disponibili.

Di conseguenza, alcuni prompt potrebbero contribuire con un numero di esempi significativamente superiore rispetto ad altri, introducendo uno sbilanciamento nella distribuzione dei dati di addestramento.

Per evitare che specifici prompt dominassero il dataset finale, è stato introdotto un limite massimo al numero di coppie generate per ciascun quesito. Quando il numero di coppie valide supera tale limite, vengono mantenute esclusivamente quelle caratterizzate dal maggiore valore di Δ.

Questa scelta consente di preservare le preferenze più informative, riducendo al contempo la ridondanza e migliorando l'equilibrio complessivo del dataset.

3.5 Costruzione dei dataset finali

L'applicazione delle strategie di estrazione delle preferenze e dei filtri di qualità descritti nelle sezioni precedenti ha portato alla generazione di diversi dataset destinati agli esperimenti di fine-tuning. L'obiettivo non era esclusivamente quello di massimizzare il numero di esempi disponibili, ma anche quello di studiare come differenti criteri di costruzione influenzino la qualità del segnale di apprendimento e le prestazioni dei modelli addestrati.

I dataset ottenuti differiscono sia per la strategia di estrazione delle preferenze adottata sia per il livello di selettività dei filtri applicati. In particolare, sono stati costruiti dataset caratterizzati da differenti compromessi tra quantità di dati disponibili e intensità del segnale di preferenza.

3.5.1 Dataset Best-vs-Worst

Il primo dataset è stato generato utilizzando la strategia Best-vs-Worst descritta nella Sezione 3.3. Per ciascun prompt è stata mantenuta esclusivamente la coppia costituita dalla risposta con il punteggio più elevato e da quella con il punteggio più basso.

Questa procedura produce un singolo esempio di addestramento per ogni prompt disponibile, privilegiando la qualità del segnale rispetto alla quantità di dati. Le coppie ottenute rappresentano infatti i casi in cui la differenza di valutazione tra le risposte risulta massima.

Il dataset finale contiene:

- Numero di prompt unici: **313**
- Numero di triple DPO generate: **313**
- Gap medio tra *chosen* e *rejected*: **1.746**
- Gap mediano tra *chosen* e *rejected*: **1.500**

3.5.2 Dataset Pairwise

A differenza della strategia Best-vs-Worst, l'approccio Pairwise consente di estrarre più coppie di preferenza per ciascun prompt. A partire dal dataset ottenuto al

termine della fase di preprocessing sono state considerate tutte le possibili coppie di risposte, successivamente filtrate applicando i criteri descritti nella Sezione 3.4.

Il dataset di partenza, prima dell'applicazione dei filtri specifici della strategia Pairwise, contiene:

- Numero di prompt unici: **313**
- Numero di triple DPO generate: **6187**
- Gap medio tra *chosen* e *rejected*: **1.256**
- Gap mediano tra *chosen* e *rejected*: **1.050**

Configurazione Pairwise ($\Delta = 0.3$)

Per la prima configurazione è stata utilizzata una soglia minima sul divario di punteggio pari a $\Delta = 0.3$, una soglia di similarità di Jaccard pari a **0.85** e un massimo di **20** coppie selezionate per ciascun prompt.

L'applicazione di tali criteri produce un dataset finale composto da **6097** triple DPO, con un gap medio pari a **1.272** e un gap mediano pari a **1.067**.

Configurazione Pairwise ($\Delta = 0.5$)

La seconda configurazione adotta criteri di selezione più restrittivi, imponendo una soglia minima pari a $\Delta = 0.5$, mantenendo invariata la soglia di similarità di Jaccard (**0.85**) e riducendo il numero massimo di coppie selezionabili per ciascun prompt da **20** a **10**.

L'applicazione congiunta di tali vincoli comporta una riduzione del numero di esempi disponibili, ottenendo un dataset finale costituito da **3058** triple DPO. Tale riduzione è dovuta sia all'aumento della soglia sul divario di punteggio, che elimina le coppie caratterizzate da preferenze meno marcate, sia al limite più restrittivo sul numero massimo di coppie conservate per ciascun prompt. Al tempo stesso, le preferenze risultano mediamente più nette, come evidenziato dall'incremento del gap medio (**1.499**) e del gap mediano (**1.300**).

3.5.3 Confronto tra i dataset

La Tabella 3.2 riassume le principali caratteristiche dei dataset generati.

Tabella 3.2. Caratteristiche dei dataset finali utilizzati negli esperimenti.

Dataset	Triple DPO	Gap medio	Gap mediano
Best-vs-Worst	313	1.746	1.500
Pairwise ($\Delta = 0.3$)	6097	1.272	1.067
Pairwise ($\Delta = 0.5$)	3058	1.499	1.300

La disponibilità di dataset caratterizzati da differenti dimensioni e differenti livelli di selettività consente di analizzare sperimentalmente l'impatto delle scelte di costruzione dei dati sul processo di fine-tuning. In particolare, gli esperimenti

descritti nei capitoli successivi permetteranno di valutare il compromesso tra quantità di esempi disponibili e qualità del segnale di preferenza fornito al modello.

3.6 Suddivisione dei dati

Dopo la generazione delle triple di preferenza, i dataset sono stati suddivisi in un insieme di addestramento (*training set*) e un insieme di test (*test set*). Tale suddivisione è stata introdotta con un duplice obiettivo: consentire l'addestramento dei modelli mediante Direct Preference Optimization e, al tempo stesso, disporre di un insieme di esempi non osservati durante il fine-tuning da utilizzare per la valutazione del comportamento del modello.

A differenza di una suddivisione casuale effettuata direttamente sulle singole triple, lo split è stato eseguito a livello di prompt. In altre parole, tutte le coppie di preferenza generate a partire dallo stesso prompt vengono assegnate interamente al training set oppure interamente al test set.

Questa scelta è stata adottata per evitare fenomeni di *data leakage*. Nel caso di una suddivisione casuale delle singole triple, sarebbe infatti possibile che esempi derivati dallo stesso prompt comparissero contemporaneamente sia nel training set sia nel test set. In tale situazione il modello potrebbe beneficiare indirettamente di informazioni già osservate durante l'addestramento, producendo una sovrastima delle prestazioni reali.

Per ciascun dataset, il 90% dei prompt unici è stato assegnato al training set, mentre il restante 10% è stato destinato al test set. La selezione è stata effettuata in maniera casuale ma deterministica mediante l'utilizzo di un seme pseudo-casuale fissato a 42, garantendo la completa riproducibilità degli esperimenti.

Il test set risultante è stato utilizzato per valutare la capacità dei modelli di riprodurre le preferenze presenti nei dati generati da AutoBench. In particolare, confrontando le prestazioni ottenute prima e dopo il fine-tuning è stato possibile misurare l'effettivo apprendimento del segnale di preferenza contenuto nei dataset costruiti.

La valutazione su questo insieme di dati rappresenta una misura interna della qualità dell'allineamento ottenuto. Essa è stata successivamente affiancata da benchmark esterni standardizzati, descritti nei capitoli successivi, al fine di verificare se l'apprendimento delle preferenze si traducesse anche in un miglioramento delle capacità generali del modello.

L'intera pipeline di costruzione dei dataset di preferenze adottata nel presente lavoro è riassunta nella Figura 3.5. A partire dal dataset grezzo prodotto da AutoBench, vengono effettuate le operazioni di pulizia e normalizzazione, seguite dall'estrazione delle coppie di preferenza, dall'applicazione dei filtri di qualità e dalla suddivisione finale nei set di addestramento e di test.

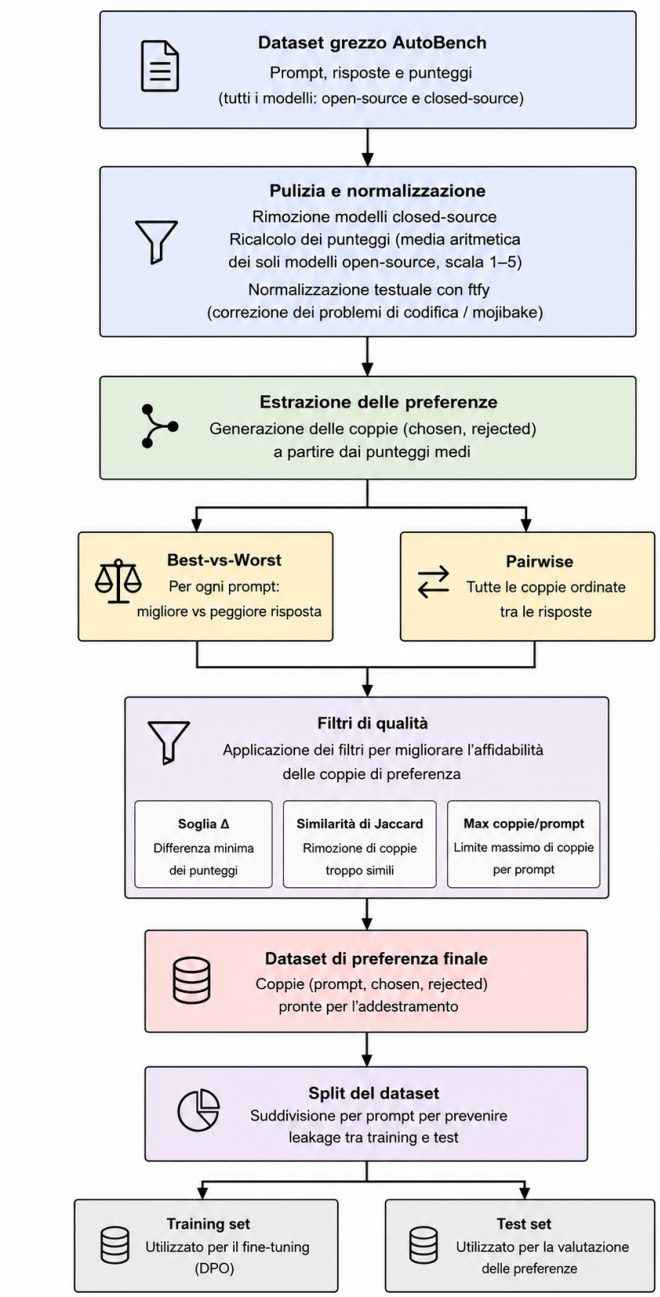


Figura 3.5. Pipeline di costruzione dei dataset di preferenze utilizzata nel presente lavoro.

Capitolo 4

Configurazione sperimentale

4.1 Obiettivi sperimentali

L'obiettivo principale del presente lavoro consiste nell'analizzare la fattibilità dell'utilizzo di dati di addestramento generati attraverso processi di valutazione tra modelli linguistici (*peer-generated training data*) per l'allineamento di Large Language Models mediante tecniche di ottimizzazione basate su preferenze.

A partire dai dati prodotti dal framework AutoBench, descritti nel Capitolo 3, sono stati costruiti diversi dataset di preferenze nel formato `prompt, chosen, rejected`, successivamente utilizzati per il fine-tuning di modelli linguistici open-source tramite l'algoritmo Direct Preference Optimization (DPO).

Gli esperimenti sono stati progettati per rispondere alle seguenti domande di ricerca:

1. È possibile utilizzare preferenze generate esclusivamente da altri modelli linguistici per effettuare il fine-tuning di un modello target senza ricorrere ad annotazioni umane?
2. In che misura il fine-tuning su dataset derivati da AutoBench modifica le prestazioni del modello rispetto alla versione di partenza?
3. Quale impatto hanno differenti strategie di costruzione delle coppie di preferenza sulla qualità del modello finale?
4. I miglioramenti osservati sul dataset derivato da AutoBench si riflettono anche su benchmark esterni comunemente utilizzati per la valutazione dei Large Language Models?

Nelle sezioni seguenti vengono descritte la configurazione sperimentale e la metodologia adottata per rispondere a tali quesiti, fornendo tutti gli elementi necessari per l'interpretazione dei risultati presentati nel Capitolo 5.

4.2 Ambiente sperimentale

Tutti gli esperimenti presentati nel presente lavoro sono stati eseguiti utilizzando la piattaforma Kaggle Notebooks, che mette a disposizione risorse computazionali

gratuite basate su GPU NVIDIA Tesla T4. Tale scelta è stata motivata dalla disponibilità immediata dell'infrastruttura e dalla possibilità di eseguire esperimenti di fine-tuning su modelli linguistici di piccole e medie dimensioni senza ricorrere a hardware dedicato.

Gli esperimenti sono stati condotti utilizzando una singola GPU NVIDIA Tesla T4 con 14,6 GB di memoria video (VRAM). A causa delle limitazioni hardware disponibili, è stato necessario adottare tecniche di ottimizzazione della memoria per rendere possibile l'addestramento dei modelli considerati.

A tal fine è stata utilizzata la libreria *Unsloth*, una raccolta di ottimizzazioni per il fine-tuning efficiente di Large Language Models che integra versioni modificate delle principali librerie dell'ecosistema Hugging Face. In particolare, Unsloth fornisce supporto nativo per il caricamento dei modelli in formato quantizzato a 4 bit, implementazioni ottimizzate di LoRA e QLoRA, oltre a meccanismi di gradient checkpointing progettati per ridurre il consumo di memoria durante l'addestramento.

Poiché la GPU Tesla T4 non supporta il formato numerico BF16 (*Brain Floating Point*), tutti gli esperimenti sono stati eseguiti utilizzando precisione FP16 (*Half Precision*), verificata automaticamente all'avvio di ciascun notebook sperimentale.

La Tabella 4.1 riassume la configurazione hardware e software utilizzata.

Tabella 4.1. Configurazione dell'ambiente sperimentale.

Componente	**Configurazione**
Piattaforma	Kaggle Notebooks
GPU	NVIDIA Tesla T4
Memoria GPU	14.6 GB VRAM
Precisione numerica	FP16
Quantizzazione	4 bit (QLoRA)
Framework principale	Unsloth
Librerie di training	TRL, Transformers, PEFT
Formato dataset	Parquet
Seed sperimentale	42

L'utilizzo congiunto della quantizzazione e delle tecniche di *Parameter-Efficient Fine-Tuning* ha reso possibile l'addestramento dei modelli considerati nel presente lavoro entro i limiti di memoria della GPU disponibile, consentendo lo svolgimento dell'intera campagna sperimentale senza ricorrere a infrastrutture di calcolo dedicate.

4.3 Procedura di fine-tuning

Tutti gli esperimenti del presente lavoro sono stati condotti utilizzando l'algoritmo Direct Preference Optimization (DPO), introdotto nel Capitolo 2.

La presente sezione descrive esclusivamente gli aspetti implementativi della procedura di fine-tuning adottata, includendo la configurazione QLoRA, gli iperparametri di addestramento e i modelli utilizzati.

4.3.1 Quantizzazione e QLoRA

Come descritto nella Sezione 2.2.4, tutti gli esperimenti sono stati eseguiti utilizzando l'approccio QLoRA tramite la libreria Unsloth su una singola GPU NVIDIA Tesla T4 (Sezione 4.2). Gli adapter sono stati applicati ai principali moduli dell'architettura Transformer, comprendendo sia i meccanismi di attenzione sia i blocchi feed-forward.

Questa configurazione permette di mantenere una buona capacità di adattamento del modello riducendo sensibilmente il numero di parametri addestrabili e, di conseguenza, il consumo di memoria durante il training.

Per tutti gli esperimenti sono stati mantenuti costanti i principali iperparametri di LoRA:

- rank (r): 16;
- fattore di scala (α): 16;
- dropout: 0;
- bias: `none`.

Tale configurazione rappresenta un buon compromesso tra capacità di adattamento del modello ed efficienza computazionale.

Un ulteriore parametro rilevante ai fini della procedura di tokenizzazione è la lunghezza massima di sequenza (`MAX_SEQ_LENGTH`), che determina il numero massimo di token considerati per ciascun esempio di addestramento, e la corrispondente lunghezza massima riservata al prompt (`MAX_PROMPT_LENGTH`), fissata in tutti gli esperimenti a metà di `MAX_SEQ_LENGTH`. Poiché le risposte generate durante il processo di *peer assessment* di AutoBench risultano spesso piuttosto lunghe, tale parametro incide direttamente sulla porzione di testo effettivamente osservata dal modello durante il training. A causa dei vincoli di memoria imposti dalla GPU Tesla T4 (Sezione 4.2), tale valore è stato configurato in maniera differente per i due modelli considerati:

- Gemma 2 2B Instruct: `MAX_SEQ_LENGTH` = 850 token, `MAX_PROMPT_LENGTH` = 425 token;
- Qwen 2.5 0.5B Instruct: `MAX_SEQ_LENGTH` = 2000 token, `MAX_PROMPT_LENGTH` = 1000 token.

Il valore più contenuto adottato per Gemma 2 2B è dovuto al maggiore ingombro in memoria di questo modello rispetto a Qwen 2.5 0.5B, che ha reso necessario un troncamento più aggressivo delle sequenze per rientrare nei limiti di VRAM disponibili. Le implicazioni di questa scelta sono discusse nella Sezione 6.5.

4.3.2 Configurazione dell'addestramento

L'addestramento è stato effettuato utilizzando l'implementazione del `DPOTrainer` fornita dalla libreria TRL [12], opportunamente ottimizzata da Unsloth mediante kernel personalizzati e gradient checkpointing avanzato.

Per tutti gli esperimenti sono stati mantenuti costanti i principali iperparametri di training:

Tabella 4.2. Iperparametri utilizzati durante il fine-tuning DPO.

Parametro	Valore
Learning rate	5×10^{-5}
Scheduler	Cosine
Ottimizzatore	AdamW 8-bit
Batch size per device	2
Gradient accumulation steps	8
Batch size effettivo	16
β DPO	0.1
Seed	42

In aggiunta, è stato impiegato il gradient checkpointing fornito da Unsloth, che consente una riduzione significativa del consumo di memoria durante il training.

Per quanto riguarda il numero di epoche di addestramento, gli esperimenti condotti sui dataset Best-vs-Worst e Pairwise ($\Delta = 0.3$) sono stati eseguiti per una singola epoca. Fa eccezione l'esperimento condotto sul dataset Pairwise ($\Delta = 0.5$), per il quale è stato invece analizzato l'andamento del modello su più checkpoint successivi, come descritto nel Capitolo 5.

4.3.3 Modelli considerati

Gli esperimenti sono stati condotti su due modelli linguistici open-source di dimensioni differenti:

- Gemma 2 2B Instruct;
- Qwen 2.5 0.5B Instruct.

La scelta di utilizzare modelli relativamente compatti è stata motivata principalmente dai vincoli computazionali disponibili. Entrambi i modelli risultano infatti compatibili con l'esecuzione su una singola GPU Tesla T4 mediante QLoRA e quantizzazione a 4 bit.

Gemma 2 2B rappresenta il modello di dimensioni maggiori utilizzato nello studio, mentre Qwen 2.5 0.5B costituisce una soluzione particolarmente leggera, che ha consentito di effettuare un numero maggiore di esperimenti su dataset differenti.

4.4 Configurazione degli esperimenti

Per analizzare l'efficacia dei dati di addestramento generati tramite *peer assessment* sono stati eseguiti diversi esperimenti di fine-tuning utilizzando combinazioni differenti di modelli e dataset di preferenze.

L'attenzione principale degli esperimenti è rivolta alla valutazione dell'impatto delle diverse strategie di costruzione del dataset di preferenze sul processo di allineamento. L'impiego di due modelli differenti consente inoltre di verificare se le tendenze osservate risultino consistenti al variare dell'architettura e della dimensione del modello.

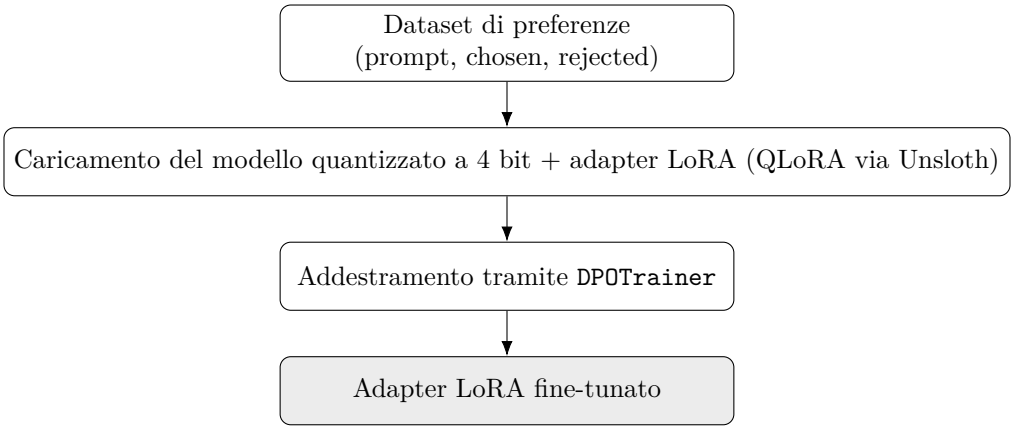


Figura 4.1. Workflow del processo di fine-tuning: il dataset di preferenze viene utilizzato da `DPOTrainer` per addestrare, tramite QLoRA, gli adapter LoRA applicati al modello base quantizzato a 4 bit, producendo come output l'adapter fine-tunato.

Gli esperimenti sono stati condotti principalmente sul modello Qwen 2.5 0.5B Instruct, scelto per le sue dimensioni contenute e per la possibilità di eseguire numerose configurazioni di training entro i limiti computazionali disponibili. In aggiunta, una parte degli esperimenti è stata replicata sul modello Gemma 2 2B Instruct al fine di verificare se le tendenze osservate risultassero consistenti anche su un modello di dimensioni maggiori.

I dataset utilizzati derivano tutti dallo stesso corpus AutoBench descritto nel Capitolo 3, ma differiscono per strategia di costruzione delle preferenze e criteri di filtraggio.

In particolare, sono stati utilizzati i tre dataset di preferenze descritti nella Sezione 3.5:

- il dataset Best-vs-Worst;
- il dataset Pairwise ottenuto con soglia $\Delta = 0.3$ e un massimo di 20 coppie per prompt;
- il dataset Pairwise ottenuto con soglia $\Delta = 0.5$ e un massimo di 10 coppie per prompt.

La Tabella 4.3 riassume gli esperimenti considerati nel presente lavoro.

Tabella 4.3. Riepilogo degli esperimenti effettuati.

Esperimento	Modello	Dataset
E1	Gemma 2 2B Instruct	Best vs Worst
E2	Qwen 2.5 0.5B Instruct	Best vs Worst
E3	Qwen 2.5 0.5B Instruct	Pairwise ($\Delta \geq 0.3$)
E4	Qwen 2.5 0.5B Instruct	Pairwise ($\Delta \geq 0.5$)

4.5 Metodologia di valutazione

La valutazione dei modelli fine-tunati è stata condotta utilizzando due approcci complementari. Da un lato sono state analizzate le prestazioni sul test set derivato da AutoBench, al fine di verificare la capacità del modello di riprodurre le preferenze presenti nei dati di addestramento; dall'altro sono stati utilizzati benchmark pubblici esterni, con l'obiettivo di misurare l'eventuale trasferimento delle conoscenze apprese verso compiti differenti.

Tale distinzione consente di valutare separatamente la capacità del modello di apprendere il segnale di preferenza contenuto nei dati e la sua eventuale capacità di generalizzazione verso task non direttamente osservati durante il processo di fine-tuning.

4.5.1 Valutazione sul test set AutoBench

Per ciascun dataset di preferenze è stato costruito un insieme di test separato mediante uno split effettuato a livello di prompt, come descritto nel Capitolo 3. Tale procedura garantisce che nessun prompt presente nel training set compaia anche nel test set, evitando possibili fenomeni di leakage.

Ogni elemento del test set è costituito da una tripla nel formato:

`prompt, chosen, rejected`

dove `chosen` rappresenta la risposta preferita secondo il processo di valutazione di AutoBench e `rejected` la risposta considerata inferiore.

La metrica principale utilizzata è l'accuracy di preferenza, definita come la percentuale di coppie `(chosen, rejected)` per le quali il modello assegna una probabilità maggiore alla risposta preferita rispetto a quella rifiutata.

Formalmente, dato un prompt x e una coppia di risposte (y^+, y^-), la previsione viene considerata corretta se:

$$\log \pi(y^+|x) > \log \pi(y^-|x)$$

dove π rappresenta il modello valutato.

L'accuracy ottenuta sul test set fornisce quindi una misura diretta della capacità del modello di apprendere le preferenze estratte dal processo di *peer assessment*.

È importante osservare che ciascun dataset di preferenze è accompagnato dal proprio insieme di test, ottenuto mediante uno split effettuato indipendentemente a livello di prompt. Di conseguenza, gli esperimenti condotti su dataset differenti vengono valutati su test set differenti. Per questo motivo i valori assoluti di accuracy ottenuti dal modello base non sono direttamente confrontabili tra esperimenti diversi; il confronto corretto riguarda invece la variazione tra il modello base e la corrispondente versione fine-tunata all'interno dello stesso esperimento.

4.5.2 Accuracy grezza e accuracy normalizzata

Poiché la probabilità complessiva assegnata ad una sequenza può essere influenzata dalla sua lunghezza, sono state considerate due differenti varianti della metrica di accuracy.

L'**accuracy grezza** confronta direttamente la probabilità totale attribuita alle due risposte candidate.

L'**accuracy normalizzata** utilizza invece la probabilità media per token, riducendo il vantaggio che potrebbe derivare da risposte particolarmente brevi o particolarmente lunghe.

L'utilizzo congiunto delle due metriche consente di distinguere eventuali miglioramenti effettivi nella qualità delle preferenze apprese da possibili effetti dovuti esclusivamente alla diversa lunghezza delle risposte generate.

Per ciascun esperimento, quando disponibile, sono state riportate entrambe le misure, consentendo un'analisi più accurata dell'impatto del fine-tuning.

4.5.3 Benchmark esterni

Sebbene l'accuracy sul test set AutoBench rappresenti una misura importante dell'apprendimento delle preferenze, essa non consente di valutare se le conoscenze acquisite si traducano in miglioramenti su compiti differenti.

Per questo motivo i modelli sono stati sottoposti a una seconda fase di valutazione mediante benchmark pubblici comunemente utilizzati nella letteratura sui Large Language Models.

I benchmark considerati includono:

- **ARC-Challenge** [13], composto da quesiti scientifici a scelta multipla progettati per valutare capacità di ragionamento e comprensione;
- **HellaSwag** [14], focalizzato sul completamento di situazioni descritte in linguaggio naturale e sulla comprensione del senso comune;
- **Winogrande** [15], utilizzato per analizzare il ragionamento basato sul contesto e la risoluzione di ambiguità linguistiche;
- **TruthfulQA** [16], progettato per valutare la capacità del modello di evitare affermazioni false ma plausibili;
- **MMLU (Massive Multitask Language Understanding)** [17], che misura la conoscenza generale del modello su numerose discipline accademiche;
- **IFEval** [18], dedicato alla valutazione della capacità di seguire istruzioni complesse.

Tali benchmark consentono di verificare se il fine-tuning basato su dati generati automaticamente produca effetti osservabili anche al di fuori del dominio specifico da cui sono state estratte le preferenze.

4.5.4 Framework di valutazione

L'esecuzione dei benchmark è stata effettuata mediante il framework *LM Evaluation Harness* [19], ampiamente utilizzato dalla comunità di ricerca per la valutazione standardizzata dei modelli linguistici.

Per ciascun modello sono state confrontate le prestazioni prima e dopo il fine-tuning, mantenendo invariata la configurazione di valutazione. In questo modo è

stato possibile attribuire eventuali variazioni nelle metriche osservate esclusivamente agli effetti del processo di allineamento.

Per ogni configurazione sperimentale vengono riportati:

- le prestazioni del modello base;
- le prestazioni del modello dopo il fine-tuning DPO;
- la variazione osservata tra le due configurazioni.

L'analisi congiunta dei risultati ottenuti sul test set AutoBench e sui benchmark esterni consente quindi di valutare sia l'apprendimento delle preferenze sia la capacità di generalizzazione dei modelli addestrati mediante *peer-generated training data*.

Capitolo 5

Risultati sperimentali

5.1 Introduzione ai risultati

In questo capitolo vengono presentati i risultati ottenuti dagli esperimenti di fine-tuning descritti nel Capitolo 4.

L'obiettivo dell'analisi è valutare l'efficacia dei dataset di preferenze costruiti a partire dai dati generati dal framework AutoBench, verificando in che misura tali dati consentano di allineare modelli linguistici tramite Direct Preference Optimization.

Per ciascun esperimento vengono confrontate le prestazioni del modello base e della corrispondente versione fine-tunata. La valutazione viene effettuata sia sul test set derivato da AutoBench, utilizzato per misurare la capacità del modello di apprendere le preferenze presenti nei dati di addestramento, sia su benchmark pubblici esterni, impiegati per analizzare l'eventuale capacità di generalizzazione delle conoscenze apprese.

Gli esperimenti sono organizzati in base al modello utilizzato e alla strategia adottata per la costruzione del dataset di preferenze. In particolare vengono presentati i risultati ottenuti con:

- il dataset Best-Worst, che associa a ciascun prompt esclusivamente la migliore e la peggiore risposta disponibili;
- il dataset Pairwise con soglia minima di preferenza pari a $\Delta = 0.3$;
- il dataset Pairwise con soglia minima di preferenza pari a $\Delta = 0.5$;

L'analisi dei risultati consentirà di evidenziare il ruolo della qualità delle coppie di preferenza e l'impatto delle diverse strategie di costruzione del dataset sulle prestazioni finali dei modelli.

5.2 Gemma 2 2B + Dataset Best-Worst

Il primo esperimento ha riguardato il modello *Gemma 2 2B Instruct*, sottoposto a fine-tuning mediante Direct Preference Optimization utilizzando il dataset di preferenze costruito con la strategia Best-Worst descritta nel Capitolo 3.

Il dataset contiene una singola coppia di preferenza per ciascun prompt, ottenuta selezionando la risposta con il punteggio più elevato e quella con il punteggio più basso tra le risposte generate durante il processo di *peer assessment*. Dopo le operazioni di filtraggio e preparazione dei dati, il dataset risultante è stato suddiviso in training set ed evaluation set, ottenendo 253 esempi per l'addestramento e 29 esempi per la valutazione interna durante il training.

Il fine-tuning è stato eseguito utilizzando la configurazione descritta nel Capitolo 4, mantenendo invariati tutti gli iperparametri principali e addestrando esclusivamente gli adapter LoRA mediante QLoRA.

5.2.1 Risultati sul test set AutoBench

La Tabella 5.1 riporta l'accuracy ottenuta sul test set derivato da AutoBench prima e dopo il fine-tuning.

Tabella 5.1. Accuracy sul test set AutoBench per Gemma 2 2B con dataset Best-Worst.

Modello	Accuracy (%)	Accuracy normalizzata (%)
Gemma 2 2B Base	54.84	67.74
Gemma 2 2B DPO	54.84	67.74

I risultati mostrano come il fine-tuning non abbia prodotto variazioni misurabili nella capacità del modello di distinguere tra risposte preferite e non preferite all'interno del test set. Sia l'accuracy standard sia la versione normalizzata rispetto alla lunghezza delle risposte rimangono infatti invariate rispetto al modello di partenza.

5.2.2 Risultati sui benchmark esterni

Per valutare l'eventuale trasferimento delle conoscenze apprese verso compiti differenti, il modello è stato inoltre sottoposto ai benchmark descritti nel Capitolo 4.

La Tabella 5.2 riassume i risultati ottenuti prima e dopo il fine-tuning.

Tabella 5.2. Prestazioni di Gemma 2 2B sui benchmark esterni.

Benchmark	Base (%)	DPO (%)	Δ
ARC-Challenge	49.20	48.80	-0.40
HellaSwag	66.00	64.40	-1.60
Winogrande	70.80	71.20	+0.40
TruthfulQA MC2	55.06	55.08	+0.02
MMLU	57.63	57.67	+0.04
IFEval	39.60	44.00	+4.40

I risultati ottenuti mostrano che il fine-tuning produce variazioni generalmente contenute sulle prestazioni del modello, confermando una sostanziale stabilità rispetto alla versione di partenza.

Le differenze osservate risultano inferiori ai due punti percentuali per la maggior parte dei benchmark. In particolare, Winogrande registra un lieve incremento pari a

0.4 punti percentuali, mentre MMLU e TruthfulQA MC2 rimangono sostanzialmente invariati.

L'incremento più evidente riguarda IFEval, che passa dal 39.6% al 44.0%, corrispondente a un miglioramento di 4.4 punti percentuali. Tale risultato suggerisce un possibile effetto positivo del fine-tuning sulla capacità del modello di seguire correttamente le istruzioni, sebbene sarebbe necessario condurre ulteriori esperimenti per verificarne la significatività.

Si osservano invece leggere diminuzioni su ARC-Challenge (-0.4 punti percentuali) e HellaSwag (-1.6 punti percentuali), entrambe comunque di entità limitata.

Nel complesso, il fine-tuning sul dataset Best-Worst mantiene pressoché inalterate le capacità generali del modello, mostrando al tempo stesso un miglioramento significativo su IFEval e una sostanziale stabilità sugli altri benchmark considerati.

5.3 Qwen 2.5 0.5B + Dataset Best-Worst

Il secondo esperimento ha riguardato il modello *Qwen 2.5 0.5B Instruct*, sottoposto a fine-tuning mediante Direct Preference Optimization utilizzando lo stesso dataset Best-Worst impiegato nell'esperimento precedente.

A differenza di Gemma 2 2B, Qwen 2.5 0.5B è un modello significativamente più compatto, contenente circa 500 milioni di parametri. Tale caratteristica lo rende particolarmente interessante per valutare l'efficacia dell'approccio proposto in scenari caratterizzati da risorse computazionali limitate.

Il fine-tuning è stato eseguito utilizzando la configurazione descritta nel Capitolo 4 e lo stesso dataset Best-Worst impiegato nell'esperimento precedente. Tale scelta consente di confrontare direttamente il comportamento di Gemma 2 2B e Qwen 2.5 0.5B a parità di dati di addestramento, evidenziando l'influenza delle caratteristiche architetturali dei due modelli sui risultati ottenuti.

5.3.1 Risultati sul test set AutoBench

La Tabella 5.3 riporta le prestazioni ottenute sul test set AutoBench prima e dopo il fine-tuning.

Tabella 5.3. Accuracy sul test set AutoBench per Qwen 2.5 0.5B con dataset Best-Worst.

Modello	Accuracy (%)	Accuracy normalizzata (%)
Qwen 2.5 0.5B Base	29.03	38.71
Qwen 2.5 0.5B DPO	29.03	38.71

Anche in questo caso il fine-tuning non produce variazioni osservabili nell'accuracy sul test set. Il modello mantiene esattamente le stesse prestazioni sia nella metrica standard sia nella versione normalizzata.

Un aspetto degno di nota riguarda il valore assoluto dell'accuracy ottenuta. Sia il modello base sia la versione fine-tunata mostrano infatti prestazioni inferiori al 50%, indicando una scarsa capacità di riprodurre le preferenze presenti nel dataset. Tale comportamento verrà discusso più approfonditamente nella Sezione6.4.

5.3.2 Risultati sui benchmark esterni

La Tabella 5.4 riporta i risultati ottenuti sui benchmark esterni.

Tabella 5.4. Prestazioni di Qwen 2.5 0.5B sui benchmark esterni.

Benchmark	Base (%)	DPO (%)	Δ
ARC-Challenge	36.40	36.40	0.00
HellaSwag	48.00	47.20	-0.80
Winogrande	59.20	58.40	-0.80
TruthfulQA MC2	42.37	42.31	-0.06
MMLU	45.92	46.11	+0.19
IFEval	20.00	16.40	-3.60

I risultati mostrano variazioni generalmente contenute. MMLU registra un lieve incremento pari a 0.19 punti percentuali, mentre ARC-Challenge rimane invariato.

Al contrario, si osservano piccole diminuzioni su HellaSwag, Winogrande e TruthfulQA. La variazione più evidente riguarda IFEval, che passa dal 20.0% al 16.4%, con una riduzione di 3.6 punti percentuali.

Nel complesso, il fine-tuning sul dataset Best-Worst non determina miglioramenti sostanziali nelle prestazioni del modello. Le variazioni osservate sui benchmark risultano infatti limitate e non evidenziano una tendenza chiara verso un incremento delle capacità generali del modello.

L'assenza di miglioramenti significativi suggerisce che, nelle condizioni sperimentali considerate, il dataset Best-Worst abbia fornito un segnale di apprendimento relativamente debole per il modello Qwen 2.5 0.5B. Le possibili cause di tale comportamento verranno discusse più approfonditamente nella Sezione6.4.

5.4 Qwen 2.5 0.5B + Dataset Pairwise ($\Delta = 0.3$)

Per valutare l'impatto della procedura di estrazione delle preferenze descritta nel Capitolo 3, è stato condotto un ulteriore esperimento utilizzando il modello Qwen 2.5 0.5B Instruct e un dataset costruito mediante la strategia Pairwise con soglia $\Delta = 0.3$.

A differenza dell'approccio Best-Worst, che seleziona esclusivamente la risposta con il punteggio più elevato e quella con il punteggio più basso per ciascun prompt, il metodo Pairwise genera una coppia di preferenza ogni volta che la differenza tra i punteggi di due risposte supera una determinata soglia. In questo modo è possibile ottenere un numero maggiore di esempi di addestramento, mantenendo al contempo un livello minimo di separazione tra risposta preferita e risposta rifiutata.

Il fine-tuning è stato effettuato utilizzando la medesima configurazione DPO descritta nel Capitolo 4.

5.4.1 Risultati sul test set AutoBench

La Tabella 5.5 riporta le prestazioni ottenute sul test set AutoBench.

A differenza di quanto osservato con il dataset Best-Worst, in questo caso si registra un lieve incremento dell'accuracy grezza, che passa dal 32.55% al 32.72%.

Tabella 5.5. Accuracy sul test set AutoBench per Qwen 2.5 0.5B con dataset Pairwise ($\Delta = 0.3$).

Modello	Accuracy (%)	Accuracy normalizzata (%)
Qwen 2.5 0.5B Base	32.55	40.94
Qwen 2.5 0.5B DPO	32.72	40.94

L'accuracy normalizzata rimane invece invariata al 40.94%, suggerendo che il miglioramento osservato nella metrica standard sia estremamente contenuto.

Nonostante tale incremento, le prestazioni complessive restano significativamente inferiori alla soglia del 50%, indicando che il modello continua a mostrare difficoltà nel riprodurre le preferenze presenti nel dataset.

5.4.2 Risultati sui benchmark esterni

La Tabella 5.6 riporta i risultati ottenuti sui benchmark esterni.

Tabella 5.6. Prestazioni di Qwen 2.5 0.5B sui benchmark esterni dopo il fine-tuning con dataset Pairwise ($\Delta = 0.3$).

Benchmark	Base (%)	DPO (%)	Δ
ARC-Challenge	36.40	36.40	0.00
HellaSwag	48.00	47.20	-0.80
Winogrande	59.20	59.20	0.00
TruthfulQA MC2	42.37	42.47	+0.10
MMLU	45.92	46.24	+0.32
IFEval	20.00	18.40	-1.60

I risultati mostrano variazioni generalmente modeste, analogamente a quanto osservato nell'esperimento Best-Worst.

L'incremento più evidente riguarda MMLU, che registra un miglioramento di 0.32 punti percentuali. Anche TruthfulQA MC2 mostra una lieve crescita (+0.10 punti), mentre ARC-Challenge e Winogrande rimangono invariati.

Le diminuzioni più rilevanti interessano HellaSwag (-0.80 punti) e IFEval (-1.60 punti), sebbene quest'ultima risulti meno marcata rispetto a quanto osservato con il dataset Best-Worst.

Nel complesso, il dataset Pairwise ($\Delta = 0.3$) sembra produrre risultati leggermente migliori rispetto all'approccio Best-Worst, sia in termini di accuracy sul test set sia per quanto riguarda alcuni benchmark esterni. Tuttavia, i miglioramenti osservati rimangono limitati e non consentono di evidenziare un incremento sostanziale delle capacità del modello.

5.5 Qwen 2.5 0.5B + Dataset Pairwise ($\Delta = 0.5$)

L'ultimo esperimento considerato nel presente lavoro utilizza nuovamente il modello *Qwen 2.5 0.5B Instruct*, addestrato mediante Direct Preference Optimization su un dataset costruito attraverso la strategia Pairwise con soglia $\Delta = 0.5$.

Come discusso nel Capitolo 3, l'aumento della soglia minima richiesta tra i punteggi delle due risposte comporta una selezione più restrittiva delle coppie di preferenza. In linea teorica, tale scelta consente di ottenere esempi caratterizzati da preferenze più marcate e quindi potenzialmente più informative per il processo di allineamento.

L'obiettivo dell'esperimento è valutare se una selezione più rigorosa delle coppie di preferenza produca effetti differenti rispetto alla configurazione Pairwise con soglia $\Delta = 0.3$.

5.5.1 Risultati sul test set AutoBench

Per analizzare l'evoluzione del modello durante il processo di addestramento, il fine-tuning è stato eseguito per un massimo di cinque epoche, salvando un checkpoint al termine di ciascuna di esse. L'accuracy è stata quindi valutata sui checkpoint intermedi per verificare l'eventuale presenza di miglioramenti progressivi.

La Tabella 5.7 riporta i risultati ottenuti per il modello base e per i checkpoint della prima e della seconda epoca. I checkpoint successivi non vengono riportati poiché non hanno mostrato variazioni apprezzabili rispetto alla seconda epoca.

Tabella 5.7. Accuracy sul test set AutoBench per Qwen 2.5 0.5B con dataset Pairwise ($\Delta = 0.5$).

Checkpoint	Accuracy (%)	Accuracy normalizzata (%)
Base	29.55	41.56
1 epoca	29.55	41.88
2 epoche	29.87	42.52

Le prestazioni sul test set mostrano un lieve miglioramento già nelle prime fasi dell'addestramento. Dopo la prima epoca l'accuracy grezza rimane invariata rispetto al modello base, mentre l'accuracy normalizzata aumenta da 41.56% a 41.88%. Al termine della seconda epoca si osserva un ulteriore incremento di entrambe le metriche, con l'accuracy che raggiunge il 29.87% e l'accuracy normalizzata il 42.52%. I checkpoint successivi non evidenziano variazioni significative rispetto a questi valori, suggerendo che il processo di ottimizzazione raggiunga rapidamente una situazione di convergenza.

5.5.2 Risultati sui benchmark esterni

La Tabella 5.8 riporta le prestazioni ottenute sui benchmark pubblici.

I risultati sui benchmark esterni mostrano variazioni contenute ma non uniformi durante il processo di addestramento. In particolare, ARC-Challenge registra una diminuzione rispetto al modello base, passando dal 36.40% al 32.00% dopo la prima epoca e recuperando parzialmente fino al 33.00% nella seconda. HellaSwag rappresenta l'unico benchmark che evidenzia un lieve miglioramento stabile, raggiungendo il 49.00% già dopo la prima epoca e mantenendo tale valore anche nella seconda. Winogrande subisce invece una riduzione di circa un punto percentuale, senza ulteriori variazioni tra i checkpoint.

Tabella 5.8. Prestazioni di Qwen 2.5 0.5B sui benchmark esterni durante il fine-tuning con Pairwise ($\Delta = 0.5$).

Benchmark	Base	1 epoca	2 epoche
ARC-Challenge	36.40	32.00	33.00
HellaSwag	48.00	49.00	49.00
Winogrande	59.20	58.00	58.00
TruthfulQA MC2	42.37	39.76	40.02
MMLU	45.92	45.84	46.07
IFEval	20.00	15.00	16.00

Anche TruthfulQA MC2 e IFEval mostrano una diminuzione rispetto al modello di partenza, con un parziale recupero nella seconda epoca che tuttavia non consente di raggiungere le prestazioni iniziali. MMLU rimane sostanzialmente stabile durante l'intero addestramento, evidenziando soltanto un lieve incremento finale, dal 45.92% al 46.07%.

Nel complesso, il fine-tuning con il dataset Pairwise ($\Delta = 0.5$) produce effetti limitati anche sui benchmark esterni. Analogamente a quanto osservato sul test set AutoBench, la maggior parte delle variazioni si manifesta già nelle prime due epoche, mentre i checkpoint successivi non evidenziano miglioramenti significativi, suggerendo una rapida convergenza del processo di ottimizzazione.

5.6 Sintesi dei risultati

Gli esperimenti presentati nelle sezioni precedenti mostrano che il fine-tuning mediante Direct Preference Optimization su dataset di preferenze generati automaticamente produce effetti generalmente contenuti sulle prestazioni dei modelli considerati.

Nel caso di Gemma 2 2B il fine-tuning mantiene sostanzialmente invariate le prestazioni sul test set AutoBench e determina soltanto lievi variazioni sui benchmark esterni. Analogamente, il modello Qwen 2.5 0.5B addestrato sul dataset Best-Worst non evidenzia miglioramenti significativi rispetto alla versione di partenza.

Il confronto tra le diverse strategie di estrazione evidenzia un pattern degno di nota: l'aumento della soglia Δ nella strategia Pairwise produce un miglioramento più netto dell'accuracy di preferenza sul test set interno, mentre l'effetto sui benchmark esterni resta contenuto e non uniforme. Questo aspetto viene approfondito nella Sezione 6.2.

Nel complesso, gli esperimenti evidenziano che i *peer-generated training data* costituiscono una sorgente utilizzabile per il fine-tuning mediante DPO, pur senza produrre, nelle configurazioni considerate, incrementi sostanziali delle prestazioni dei modelli.

Le possibili cause di questi risultati, le implicazioni metodologiche emerse durante la sperimentazione e i limiti dell'approccio proposto vengono discussi nel capitolo seguente.

Capitolo 6

Discussione dei risultati

6.1 Valutazione dell'efficacia dei peer-generated training data

L'obiettivo principale del presente lavoro era valutare la possibilità di utilizzare dati di preferenza generati automaticamente da altri Large Language Models come sorgente di supervisione per il fine-tuning mediante Direct Preference Optimization, senza ricorrere ad annotazioni umane.

Nel complesso, i risultati ottenuti mostrano che tale approccio risulta tecnicamente praticabile. È stato infatti possibile costruire dataset di preferenze a partire dai punteggi prodotti dal framework AutoBench, utilizzarli per il fine-tuning di modelli linguistici open-source e valutarne sistematicamente gli effetti sia su un test set derivato dallo stesso framework sia mediante benchmark pubblici standardizzati.

Dal punto di vista delle prestazioni, gli esperimenti non evidenziano miglioramenti marcati rispetto ai modelli di partenza. Nella maggior parte delle configurazioni considerate, il fine-tuning produce variazioni contenute sia nell'accuracy di preferenza sia nei benchmark esterni, indicando che l'utilizzo di dataset relativamente piccoli e costruiti automaticamente non è sufficiente, da solo, a modificare in modo sostanziale il comportamento del modello.

Tuttavia, l'assenza di miglioramenti significativi non implica necessariamente che il paradigma dei *peer-generated training data* sia inefficace. Al contrario, alcuni esperimenti mostrano come dataset caratterizzati da preferenze più affidabili producano effetti leggermente migliori rispetto a quelli ottenuti mediante strategie di estrazione più semplici. Questo suggerisce che la qualità del segnale di preferenza rappresenti un fattore determinante per il successo del processo di allineamento.

Un ulteriore elemento emerso dagli esperimenti riguarda la sostanziale stabilità delle prestazioni dei modelli sui benchmark esterni. Nella maggior parte dei casi il fine-tuning non determina un degrado significativo delle capacità generali del modello, indicando che l'aggiornamento tramite DPO, almeno nelle configurazioni considerate, tende a modificare il comportamento del modello in maniera relativamente conservativa.

Nel complesso, i risultati ottenuti indicano quindi che i dati di preferenza generati automaticamente costituiscono una possibile sorgente di supervisione per l'allineamento di Large Language Models. Allo stesso tempo, essi evidenziano come

l'efficacia di tale approccio dipenda in larga misura dalla qualità delle preferenze utilizzate, suggerendo che la costruzione del dataset rappresenti un aspetto altrettanto importante quanto la scelta dell'algoritmo di ottimizzazione impiegato per il fine-tuning.

6.2 Influenza della qualità del dataset di preferenze

Uno degli aspetti emersi con maggiore evidenza durante la campagna sperimentale riguarda l'importanza della qualità del dataset di preferenze utilizzato per il fine-tuning. Pur adottando la stessa procedura di addestramento, gli stessi iperparametri e i medesimi modelli di partenza, i risultati ottenuti mostrano differenze riconducibili principalmente alle caratteristiche del dataset impiegato.

Gli esperimenti condotti nel presente lavoro hanno permesso di confrontare differenti strategie di costruzione delle coppie di preferenza. In particolare, sono stati considerati un dataset ottenuto mediante selezione Best-Worst e due dataset Pairwise costruiti imponendo differenti soglie minime sul divario di punteggio tra le risposte ($\Delta = 0.3$ e $\Delta = 0.5$). Tutti i dataset sono stati generati seguendo la medesima pipeline di elaborazione descritta nel Capitolo 3 e utilizzati mantenendo invariata la configurazione sperimentale, consentendo così un confronto diretto tra le diverse strategie di estrazione delle preferenze.

I risultati mostrano come il dataset Best-Worst e il dataset Pairwise con soglia $\Delta = 0.3$ producano effetti generalmente limitati sulle prestazioni dei modelli. Le variazioni osservate sia nell'accuracy di preferenza sia nei benchmark esterni risultano infatti contenute e, nella maggior parte dei casi, comprese entro pochi punti percentuali.

Il confronto con il dataset Pairwise costruito utilizzando una soglia più elevata ($\Delta = 0.5$) evidenzia invece miglioramenti leggermente più marcati su alcune metriche di valutazione. Sebbene tali incrementi rimangano modesti, essi suggeriscono che una selezione più restrittiva delle coppie di preferenza possa fornire un segnale di apprendimento maggiormente informativo rispetto a quello ottenuto mediante confronti tra risposte qualitativamente molto simili.

Una possibile spiegazione risiede nella natura delle coppie utilizzate durante il fine-tuning. Quando il divario qualitativo tra due risposte è ridotto, entrambe possono risultare plausibili oppure differire soltanto per aspetti marginali. In queste situazioni il modello riceve un segnale di preferenza relativamente debole, che rende più difficile apprendere un comportamento significativamente diverso da quello iniziale. Al contrario, coppie caratterizzate da differenze qualitative più evidenti forniscono indicazioni più chiare riguardo alla risposta da preferire, facilitando il processo di ottimizzazione.

I risultati ottenuti suggeriscono quindi che la semplice disponibilità di un numero elevato di esempi non sia sufficiente a garantire un allineamento efficace. La qualità delle preferenze utilizzate durante il fine-tuning sembra infatti rivestire un ruolo almeno altrettanto importante della quantità di dati disponibili.

Questa osservazione assume particolare rilevanza nel contesto dei *peer-generated training data*. A differenza delle preferenze raccolte da annotatori umani, i giudizi prodotti automaticamente da altri modelli linguistici possono risentire di errori

di valutazione, disaccordi tra i modelli coinvolti o rumore introdotto durante il processo di *peer assessment*. Di conseguenza, la fase di costruzione del dataset assume un'importanza centrale: procedure di filtraggio, selezione e validazione delle coppie di preferenza possono incidere significativamente sulla qualità del segnale utilizzato durante il fine-tuning.

Nel complesso, gli esperimenti condotti nel presente lavoro suggeriscono quindi che il principale fattore limitante non sia rappresentato dall'algoritmo DPO, che si è dimostrato stabile in tutte le configurazioni considerate, bensì dalla qualità dell'informazione fornita durante l'addestramento. Migliorare le strategie di costruzione dei dataset di preferenze appare pertanto una delle direzioni più promettenti per aumentare l'efficacia dei metodi di allineamento basati su dati generati automaticamente.

6.3 Interpretazione critica dei risultati sperimentali

I risultati presentati nel Capitolo 5 mostrano come il fine-tuning mediante Direct Preference Optimization produca, nella maggior parte degli esperimenti, variazioni contenute sia nell'accuracy sul test set AutoBench sia nei benchmark pubblici utilizzati per la valutazione. Sebbene tali miglioramenti possano apparire limitati, essi risultano coerenti con le caratteristiche della configurazione sperimentale adottata nel presente lavoro.

Un primo elemento da considerare riguarda la dimensione dei dataset di addestramento. I dataset di preferenze costruiti a partire dai dati di AutoBench contengono infatti un numero relativamente ridotto di esempi rispetto ai dataset comunemente utilizzati per l'allineamento dei Large Language Models. In particolare, il dataset Best-Worst comprende circa trecento triple di preferenza, mentre i dataset Pairwise raggiungono alcune migliaia di esempi. Pur essendo sufficienti per studiare l'effetto di differenti strategie di costruzione delle preferenze, tali quantità limitano inevitabilmente l'entità delle modifiche che il modello può apprendere durante il fine-tuning.

Un secondo aspetto riguarda la natura stessa di Direct Preference Optimization. A differenza di tecniche di addestramento più aggressive, DPO è progettato per modificare il comportamento del modello esclusivamente sulla base delle preferenze osservate, mantenendo al tempo stesso una forte vicinanza alla distribuzione appresa durante il pre-training. Questo comportamento conservativo rappresenta uno dei principali punti di forza dell'algoritmo, poiché riduce il rischio di degradare le capacità generali del modello, ma comporta anche miglioramenti generalmente graduali, soprattutto quando il segnale di preferenza disponibile è limitato.

Anche la scelta di utilizzare tecniche di *Parameter-Efficient Fine-Tuning* contribuisce a spiegare i risultati osservati. In tutti gli esperimenti sono stati aggiornati esclusivamente gli adapter LoRA, mantenendo congelati i parametri del modello base. Tale approccio consente di ridurre drasticamente il fabbisogno computazionale e di rendere possibile l'addestramento su hardware limitato, ma offre inevitabilmente una capacità di adattamento inferiore rispetto a un fine-tuning completo dell'intero modello.

Infine, gli esperimenti sono stati condotti su modelli di dimensioni relativamente contenute, Gemma 2 2B Instruct e Qwen 2.5 0.5B Instruct. Sebbene tali modelli rappresentino un'interessante scelta per valutare la fattibilità dell'approccio in contesti accessibili, essi dispongono di una capacità rappresentativa inferiore rispetto ai Large Language Models di dimensioni maggiori, rendendo plausibile che l'effetto del fine-tuning risulti meno marcato.

Alla luce di queste considerazioni, i risultati ottenuti appaiono coerenti con il contesto sperimentale del presente lavoro. In particolare, il fatto che il fine-tuning non abbia prodotto variazioni sostanziali sui benchmark esterni, pur mantenendo generalmente stabili le prestazioni del modello, suggerisce che l'utilizzo di dati di preferenza generati automaticamente costituisca un approccio promettente ma che richieda dataset sufficientemente estesi e accurati per produrre miglioramenti più evidenti. I risultati ottenuti evidenziano quindi l'importanza non solo della disponibilità di dati di preferenza, ma anche della loro qualità e della loro capacità di fornire un segnale informativo sufficientemente forte durante il processo di allineamento.

6.4 Bias osservati durante la valutazione

6.4.1 Bias di lunghezza

Nel corso della valutazione dei modelli è emerso un aspetto metodologico che ha influenzato in modo significativo l'interpretazione dei risultati ottenuti sul test set AutoBench. In particolare, durante i primi esperimenti è stato osservato che il modello Qwen 2.5 0.5B otteneva un'accuracy di preferenza sistematicamente inferiore al 50%, sia prima sia dopo il fine-tuning.

Un risultato di questo tipo appariva difficilmente giustificabile. L'accuracy di preferenza viene infatti calcolata confrontando, per ciascun prompt, la probabilità assegnata dal modello alla risposta *chosen* e a quella *rejected*. Un valore stabilmente inferiore al 50% suggerirebbe che il modello preferisca sistematicamente la risposta ritenuta peggiore dal dataset, ottenendo prestazioni inferiori a quelle attese da una scelta casuale.

Per comprendere l'origine di questo comportamento è stata analizzata la modalità con cui venivano confrontate le log-probabilità delle due risposte. Nella formulazione iniziale dell'accuracy veniva infatti utilizzata la somma delle log-probabilità dei token che compongono ciascuna risposta. Poiché tale quantità cresce in valore assoluto con l'aumentare della lunghezza della sequenza, risposte più lunghe tendono naturalmente ad accumulare log-probabilità più negative rispetto a risposte più brevi, indipendentemente dalla loro qualità.

Per ridurre questo effetto è stata introdotta una seconda metrica, ottenuta normalizzando la log-probabilità complessiva rispetto al numero di token della risposta. In questo modo il confronto viene effettuato utilizzando la log-probabilità media per token, limitando l'influenza della diversa lunghezza delle sequenze generate.

L'introduzione di questa normalizzazione ha prodotto un incremento dell'accuracy osservata per entrambi i modelli. Nel caso di Gemma 2 2B l'accuracy è passata dal 54.84% al 67.74%, mentre per Qwen 2.5 0.5B si è osservato un miglioramento compreso tra circa il 39% e il 42%, a seconda del dataset di valutazione utilizzato. Pur rimanendo inferiore al 50%, quest'ultimo risultato evidenzia comunque come

una parte rilevante delle prestazioni inizialmente osservate fosse riconducibile al bias introdotto dalla lunghezza delle risposte.

È importante sottolineare che la normalizzazione non modifica il comportamento del modello né le prestazioni ottenute sui benchmark esterni, ma interviene esclusivamente sulla modalità di valutazione dell'accuracy di preferenza. Essa rappresenta quindi una misura volta a rendere il confronto tra risposte di diversa lunghezza maggiormente equo.

L'osservazione di questo fenomeno suggerisce inoltre una riflessione più generale sulla valutazione di modelli addestrati mediante dataset di preferenze. Quando le risposte confrontate presentano lunghezze significativamente differenti, l'utilizzo diretto della log-probabilità totale può introdurre un bias sistematico nella metrica di valutazione. Sebbene la normalizzazione per lunghezza non elimini completamente questo problema, essa costituisce un accorgimento semplice ed efficace per ridurne l'impatto e ottenere una misura più rappresentativa delle reali preferenze apprese dal modello.

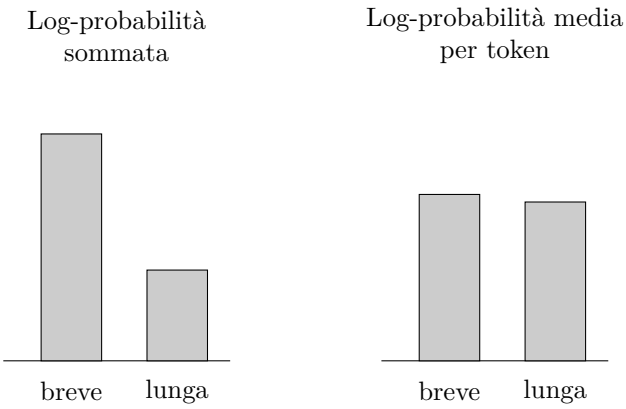


Figura 6.1. Illustrazione qualitativa dell'effetto della normalizzazione: con la log-probabilità sommata (sinistra) la risposta più lunga presenta una penalizzazione nettamente maggiore, indipendentemente dalla sua qualità; normalizzando per il numero di token (destra) le due risposte diventano confrontabili.

6.4.2 Un'ipotesi aggiuntiva: bias di tipicità

La normalizzazione per lunghezza corregge gran parte della distorsione introdotta dal confronto tra risposte di lunghezza differente, ma non è sufficiente a spiegare interamente il comportamento osservato per Qwen 2.5 0.5B, la cui accuracy di preferenza normalizzata resta stabilmente al di sotto della soglia del 50% (38.71%–42.52% a seconda del dataset), a fronte del 67.74% ottenuto da Gemma 2 2B. Se il bias di lunghezza fosse l'unico fattore in gioco, ci si aspetterebbe che la normalizzazione riportasse l'accuracy quantomeno in prossimità del livello di una scelta casuale; lo scarto residuo suggerisce la presenza di un secondo fattore.

Una possibile spiegazione trova riscontro in un filone di ricerca che studia la relazione tra la probabilità assegnata da un modello linguistico a una sequenza e la sua qualità percepita. Meister et al. [20] osservano che le stringhe a probabilità massima secondo un modello linguistico non coincidono generalmente con quelle

giudicate migliori da un lettore umano, tendendo piuttosto a risultare generiche e prevedibili. Tan et al. [21] estendono l'osservazione ai modelli allineati tramite RLHF, mostrando che, superata una certa soglia, un'ulteriore crescita della probabilità assegnata a una risposta è associata a una diminuzione della qualità percepita. Sebbene tale risultato riguardi RLHF, la stessa dinamica appare plausibile anche per DPO, che condivide con RLHF l'obiettivo di massimizzare, entro un vincolo di regolarizzazione KL, la probabilità delle risposte preferite.

Applicata al caso qui discusso, questa relazione fornisce una possibile chiave di lettura: le risposte *chosen* nel dataset AutoBench sono selezionate tramite il giudizio collettivo di un pool eterogeneo di modelli *peer*, un processo che tende a premiare risposte più originali o distintive rispetto alla media dei generatori coinvolti, mentre le risposte *rejected* possono risultare più generiche. Se le risposte scelte sono, in aggregato, meno "tipiche" secondo la distribuzione di un modello di piccole dimensioni, tale modello tenderà ad assegnare loro una probabilità inferiore ancora prima di qualunque fine-tuning – un effetto coerente con il fenomeno di *likelihood bias* documentato anche nella valutazione automatica tramite LLM-as-a-Judge da Oi et al. [22].

Non è possibile, sulla base dei soli dati raccolti, stabilire con certezza se e quanto la scala del modello moduli questo effetto: il confronto tra Gemma 2 2B e Qwen 2.5 0.5B coinvolge una differenza di parametri troppo ampia, e differenze nel corpus di pre-training o nella tokenizzazione potrebbero contribuire in egual misura. Quella qui presentata resta pertanto un'ipotesi interpretativa plausibile e coerente con la letteratura, non una dimostrazione diretta: una sua verifica – ad esempio confrontando la tipicità stilistica delle risposte *chosen* e *rejected*, o ripetendo l'esperimento su modelli di scala intermedia – viene proposta come direzione di ricerca futura nella Sezione 6.6.

Ciò che l'osservazione suggerisce con maggiore certezza è un limite metodologico più generale: l'accuracy di preferenza calcolata tramite log-probabilità, anche una volta normalizzata per lunghezza, non è una metrica neutra rispetto alla tipicità stilistica delle risposte confrontate, specialmente quando il modello valutato ha una capacità rappresentativa limitata rispetto al pool di modelli che ha generato le preferenze di riferimento. Questo non inficia tuttavia il confronto relativo tra modello base e modello fine-tunato all'interno dello stesso esperimento, che resta la misura più affidabile dell'effetto del training.

6.5 Limiti del presente lavoro

Sebbene gli esperimenti condotti abbiano permesso di analizzare il comportamento del fine-tuning mediante dati di preferenza generati automaticamente, il presente lavoro presenta alcune limitazioni che devono essere considerate nell'interpretazione dei risultati.

Una prima limitazione riguarda il numero di modelli valutati. Gli esperimenti sono stati infatti condotti esclusivamente su Gemma 2 2B Instruct e Qwen 2.5 0.5B Instruct, entrambi appartenenti alla categoria dei Large Language Models di dimensioni relativamente contenute. Sebbene tale scelta abbia consentito di effettuare numerosi esperimenti su hardware accessibile, non è possibile estendere automati-

camente le conclusioni ottenute a modelli di dimensioni maggiori o appartenenti a famiglie differenti.

Un secondo limite riguarda il numero e la varietà dei dataset di preferenze considerati. Nel presente lavoro sono state confrontate differenti strategie di costruzione delle coppie di preferenza, tra cui Best-Worst e Pairwise con soglie differenti, ma tutte derivano dalla medesima sorgente di dati, rappresentata da AutoBench. Sarebbe pertanto interessante verificare se le stesse osservazioni risultino valide anche utilizzando dataset ottenuti mediante differenti procedure di generazione automatica delle preferenze.

Dal punto di vista computazionale, gli esperimenti sono stati eseguiti utilizzando una singola GPU NVIDIA Tesla T4 con 14.6 GB di memoria video. Sebbene l'impiego di QLoRA e della quantizzazione a 4 bit abbia reso possibile il fine-tuning dei modelli selezionati, tale configurazione ha inevitabilmente limitato le dimensioni dei modelli utilizzabili, il numero di esperimenti eseguibili e la possibilità di esplorare configurazioni di addestramento più complesse.

Questo stesso vincolo di memoria ha reso necessario limitare la lunghezza massima delle sequenze tokenizzate durante il training (Sezione 2.2.4), fissata a 850 token per Gemma 2 2B Instruct e a 2000 token per Qwen 2.5 0.5B Instruct, con un limite più stringente per il primo dovuto al suo maggiore ingombro in memoria. Poiché le risposte prodotte durante il processo di *peer assessment* di AutoBench risultano spesso piuttosto lunghe, tali soglie possono aver comportato il troncamento di una parte del contenuto delle risposte più estese, in particolare nel caso di Gemma 2 2B. Un troncamento di questo tipo rischia di alterare il segnale di preferenza osservato dal modello durante il training, ad esempio recidendo porzioni di risposta rilevanti ai fini del confronto tra `chosen` e `rejected`, e costituisce quindi un ulteriore fattore, di natura puramente infrastrutturale, che può aver contribuito a limitare l'efficacia del fine-tuning osservata negli esperimenti condotti su questo modello.

Un ulteriore limite è rappresentato dall'assenza di un confronto diretto con dataset di preferenze annotate da esseri umani. L'obiettivo del presente lavoro non era infatti confrontare il paradigma dei *peer-generated training data* con il tradizionale Reinforcement Learning from Human Feedback, bensì valutare la fattibilità dell'utilizzo di preferenze generate esclusivamente da altri modelli linguistici. Di conseguenza, i risultati ottenuti non consentono di stabilire quanto le preferenze automatiche si avvicinino, in termini di efficacia, a quelle prodotte da annotatori umani.

Infine, la valutazione è stata condotta mediante un insieme limitato di benchmark pubblici e attraverso l'accuracy sul test set derivato da AutoBench. Sebbene tali metriche rappresentino strumenti consolidati per la valutazione dei Large Language Models, esse non esauriscono tutti gli aspetti del comportamento di un modello linguistico, quali la robustezza, la capacità di generalizzazione su domini specialistici o la qualità percepita dagli utenti durante interazioni reali.

Nel complesso, tali limitazioni non riducono l'interesse dei risultati presentati, ma delineano con maggiore precisione il contesto entro cui essi devono essere interpretati. Il presente lavoro si propone infatti come uno studio esplorativo volto a valutare la fattibilità dell'impiego di dati di preferenza generati automaticamente per l'allineamento di Large Language Models, fornendo una base sperimentale sulla quale sviluppare future analisi più ampie e approfondite.

6.6 Prospettive future

I risultati ottenuti nel presente lavoro evidenziano diverse possibili direzioni di ricerca che potrebbero contribuire a chiarire ulteriormente il potenziale dei *peer-generated training data* per l'allineamento dei Large Language Models.

Una prima prospettiva riguarda l'utilizzo di dataset di preferenze più estesi. Gli esperimenti hanno mostrato come la qualità del dataset influenzi in modo significativo le prestazioni finali del modello; tuttavia, il numero di esempi disponibili rimane relativamente contenuto rispetto agli standard attuali dell'allineamento dei LLM. L'applicazione della pipeline proposta a versioni più ampie di AutoBench potrebbe consentire di valutare se gli effetti osservati diventino più evidenti all'aumentare della quantità di dati disponibili.

Un secondo sviluppo consiste nell'estendere la sperimentazione a modelli linguistici di dimensioni maggiori. Sebbene Gemma 2 2B e Qwen 2.5 0.5B abbiano rappresentato una scelta adeguata per gli obiettivi del presente lavoro, modelli con un numero superiore di parametri potrebbero beneficiare in misura diversa del fine-tuning mediante preferenze generate automaticamente. Un confronto sistematico tra modelli appartenenti a differenti famiglie e scale dimensionali permetterebbe di valutare quanto le conclusioni ottenute siano generalizzabili.

Un ulteriore ambito di ricerca riguarda il miglioramento della qualità delle preferenze automatiche. Nel presente lavoro i dataset sono stati costruiti a partire dai punteggi prodotti dal processo di *peer assessment* di AutoBench, ma potrebbero essere investigate strategie più sofisticate per la selezione delle coppie di preferenza, ad esempio combinando il giudizio di modelli differenti, introducendo criteri di filtraggio aggiuntivi o assegnando un peso diverso alle preferenze in funzione del livello di accordo tra i modelli valutatori.

Le osservazioni emerse durante la valutazione suggeriscono inoltre l'opportunità di approfondire gli aspetti metodologici legati alla costruzione e alla valutazione dei dataset di preferenze. In particolare, il bias di lunghezza osservato nel calcolo dell'accuracy evidenzia come la scelta della metrica possa influenzare sensibilmente l'interpretazione dei risultati. Lo sviluppo di metriche maggiormente robuste o di procedure di valutazione alternative potrebbe contribuire a ottenere stime più affidabili della capacità dei modelli di apprendere le preferenze.

Infine, un'estensione naturale del presente lavoro consiste nel confrontare direttamente dataset di preferenze generate automaticamente con dataset costruiti mediante annotazione umana. Un'analisi di questo tipo permetterebbe di valutare non solo le differenze nelle prestazioni finali dei modelli, ma anche il compromesso tra qualità del segnale di preferenza, costi di raccolta dei dati e scalabilità del processo di allineamento.

Nel complesso, il presente lavoro rappresenta un primo passo nell'analisi dell'utilizzo di dati di preferenza generati automaticamente per il fine-tuning di Large Language Models. Sebbene i risultati ottenuti mostrino miglioramenti contenuti, essi confermano la fattibilità dell'approccio e suggeriscono che la qualità dei dati di preferenza costituisca un fattore determinante per l'efficacia dell'allineamento. L'evoluzione dei framework di valutazione automatica e la crescente disponibilità di modelli linguistici sempre più avanzati rendono plausibile che i *peer-generated*

training data possano assumere un ruolo sempre più rilevante nello sviluppo di future tecniche di allineamento.

Capitolo 7

Conclusioni

Il presente lavoro ha analizzato la possibilità di utilizzare dati di preferenza generati automaticamente attraverso processi di *peer assessment* tra Large Language Models come sorgente di dati per il fine-tuning mediante *Direct Preference Optimization* (DPO), senza ricorrere ad annotazioni umane.

A partire dai dati prodotti dal framework AutoBench è stata sviluppata una pipeline completa per la costruzione di dataset di preferenze nel formato `(prompt, chosen, rejected)`. La pipeline comprende le fasi di preparazione dei dati, filtraggio, deduplicazione ed estrazione delle coppie di preferenza secondo differenti strategie, consentendo di trasformare le valutazioni prodotte automaticamente dai modelli in dati utilizzabili per l'allineamento di altri modelli linguistici.

La validità dell'approccio è stata valutata attraverso una campagna sperimentale condotta su due Large Language Models open-source, Gemma 2 2B Instruct e Qwen 2.5 0.5B Instruct, utilizzando diversi dataset di preferenze costruiti a partire dagli stessi dati di partenza ma mediante differenti criteri di selezione delle coppie. Le prestazioni dei modelli sono state analizzate sia mediante un test set derivato da AutoBench sia attraverso benchmark pubblici standardizzati, consentendo di valutare contemporaneamente l'apprendimento delle preferenze e la capacità di generalizzazione.

I risultati ottenuti mostrano che il fine-tuning basato su *peer-generated training data* è effettivamente realizzabile e non determina un degrado significativo delle capacità generali dei modelli. Allo stesso tempo, gli esperimenti evidenziano come i miglioramenti ottenuti risultino generalmente contenuti e dipendano in misura rilevante dalla qualità del dataset di preferenze utilizzato. In particolare, la selezione di coppie caratterizzate da differenze di punteggio più marcate tende a produrre un segnale di apprendimento più efficace rispetto a strategie meno selettive — un effetto osservato con maggiore chiarezza sull'accuracy di preferenza interna che sui benchmark esterni — confermando che la qualità del segnale di preferenza rappresenta un elemento almeno tanto centrale quanto la quantità di dati per l'efficacia del processo di allineamento.

Durante l'attività sperimentale è stato inoltre individuato un bias legato alla lunghezza delle risposte nel calcolo delle log-probabilità, che ha influenzato la valutazione dell'accuracy di preferenza, in particolare nel caso del modello Qwen 2.5 0.5B. Una volta corretto tale bias, è emerso come un secondo fattore, legato alla

tipicità stilistica delle risposte rispetto alla distribuzione appresa dal modello, possa contribuire a spiegare il divario residuo tra i due modelli considerati; si tratta tuttavia di un'ipotesi interpretativa che richiederebbe verifiche mirate per essere confermata. L'analisi di questi fenomeni ha comunque evidenziato l'importanza di utilizzare metriche di valutazione adeguate e ha permesso di interpretare correttamente risultati che, in una prima analisi, apparivano difficilmente spiegabili.

Sebbene il presente lavoro sia stato condotto su un numero limitato di modelli e di dataset, i risultati ottenuti forniscono indicazioni interessanti sul potenziale dei dati di preferenza generati automaticamente. In particolare, emerge come il principale fattore limitante non sia l'assenza di annotazioni umane, bensì la qualità delle preferenze estratte e la capacità del processo di valutazione di produrre confronti sufficientemente affidabili e informativi.

In conclusione, il presente lavoro conferma la fattibilità dell'utilizzo di *peer-generated training data* come possibile alternativa, almeno in alcuni contesti, ai tradizionali dataset di preferenze umane per il fine-tuning di Large Language Models. Pur non rappresentando un sostituto completo delle annotazioni umane, tale approccio offre interessanti prospettive in termini di scalabilità, costi di raccolta dei dati e automatizzazione del processo di allineamento. L'evoluzione dei framework di valutazione automatica e il miglioramento dei modelli utilizzati come giudici potrebbero rendere questo paradigma una componente sempre più rilevante nello sviluppo dei futuri sistemi di allineamento dei Large Language Models.

Bibliografia

[1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[2] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

[3] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized llms. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

[4] Daniel Han, Michael Han, and Unsloth team. Unsloth, 2023. https://github.com/unslothai/unsloth.

[5] Ralph Allan Bradley and Milton E. Terry. Rank analysis of incomplete block designs: I. the method of paired comparisons. *Biometrika*, 39(3/4):324–345, 1952.

[6] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[7] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

[8] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging llm-as-a-judge with mt-bench and chatbot arena. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

[9] Ruosen Li, Teerth Patel, and Xinya Du. Prd: Peer rank and discussion improve large language model based evaluations. *Transactions on Machine Learning Research*, 2024.

[10] Ganqu Cui, Lifan Yuan, Ning Ding, Guanming Yao, Bingxiang He, Wei Zhu, Yuan Ni, Guotong Xie, Ruobing Xie, Yankai Lin, Zhiyuan Liu, and Maosong Sun. ULTRAFEEDBACK: Boosting language models with scaled AI feedback. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 9722–9744. PMLR, 2024.

[11] Dario Loi, Elena Maria Muià, Federico Siciliano, Giovanni Trappolini, Vincenzo Crisà, Peter Kruger, and Fabrizio Silvestri. Autobench: Automating llm evaluation through reciprocal peer assessment. *arXiv preprint arXiv:2510.22593*, 2025.

[12] Leandro von Werra, Younes Belkada, Lewis Tunstall, Edward Beeching, Tristan Thrush, Nathan Lambert, Shengyi Huang, Kashif Rasul, and Quentin Gallouédec. TRL: Transformer reinforcement learning, 2020. https://github.com/huggingface/trl.

[13] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018.

[14] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. HellaSwag: Can a machine really predict the next event? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 4791–4800, 2019.

[15] Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. Winogrande: An adversarial winograd schema challenge at scale. *Communications of the ACM*, 64(9):99–106, 2021.

[16] Stephanie Lin, Jacob Hilton, and Owain Evans. TruthfulQA: Measuring how models mimic human falsehoods. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3214–3252, 2022.

[17] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *International Conference on Learning Representations*, 2021.

[18] Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. Instruction-following evaluation for large language models. *arXiv preprint arXiv:2311.07911*, 2023.

[19] Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac'h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. A framework for few-shot language model evaluation, 12 2023.

[20] Clara Meister, Tiago Pimentel, Gian Wiher, and Ryan Cotterell. Locally typical sampling. *Transactions of the Association for Computational Linguistics*, 11:102–121, 2023.

[21] Naaman Tan, Josef Valvoda, Tianyu Liu, Anej Svete, Yanxia Qin, Min-Yen Kan, and Ryan Cotterell. A probability–quality trade-off in aligned language models and its relation to sampling adaptors. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 14805–14829, 2024.

[22] Masanari Oi, Masahiro Kaneko, Ryuto Koike, Mengsay Loem, and Naoaki Okazaki. Likelihood-based mitigation of evaluation bias in large language models. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 3237–3245, 2024.